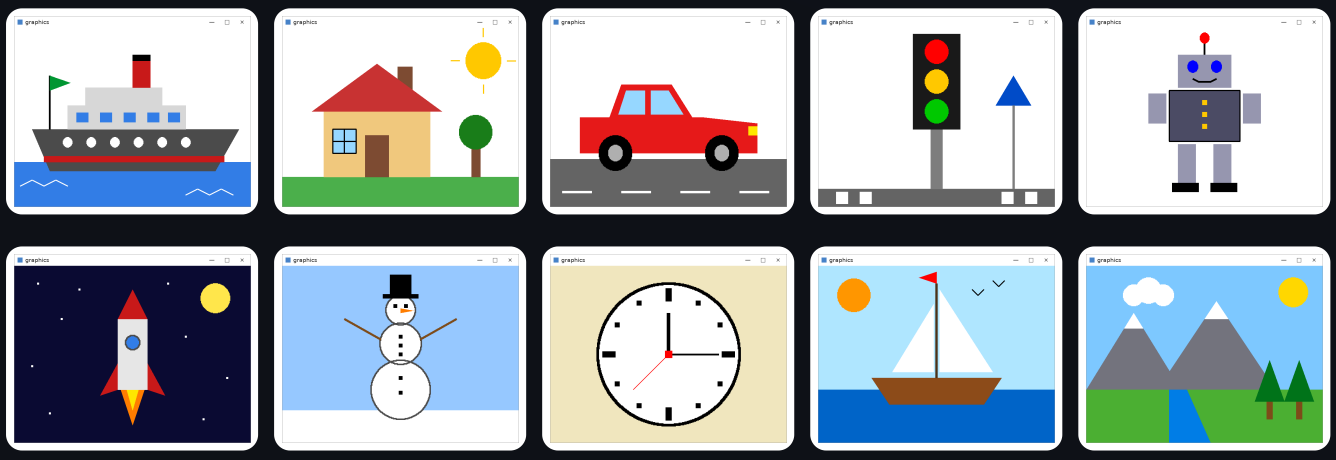
CMPN322 · MID TERM WRITTEN EXAM · PRACTICE BOOK

Computer Graphics Practice Models

Ten mid-term models in the same layout as the Fall 2025 paper, a solving guide for every question type, full model answers and a quick answer key. Every method comes from Lectures 1–6 and Sections 1–7, with the page shown.

Part 1 · Solving guide | Part 2 · 10 exam papers | Part 3 · Model answers | Part 4 · Answer key



10	160	20	10
Exam models	Theory items	Problems	Programs

Contents

Solve the paper first, then open its model answer

01	**Part 1 — Solving Guide** · how to solve each question type	03
02	**Part 2 — Exam Papers** · 10 models, questions only	14
03	**Part 3 — Model Answers**	35
04	**Part 4 — Quick Answer Key**	112

MODEL	Q3 FIGURE	QUESTION 2	PAPER	ANSWER
M1	Ship	Midpoint ellipse · P′ = T(6, 3) · R(45) · S(3, 1) · T(−6, −3) · P	14	35
M2	House	Midpoint circle · P′ = R(30) · T(5, −2) · S(0.5, 0.5) · P	16	44
M3	Car	Bresenham line · P′ = T(−2, 4) · S(1, −1) · R(−60) · P	18	52
M4	Traffic Light	Midpoint ellipse · P′ = T(3, 3) · R(90) · T(−3, −3) · T(4, 0) · P	20	59
M5	Robot	DDA line · P′ = S(2, 2) · T(1, 1) · R(180) · P	22	67
M6	Rocket	Midpoint ellipse · P′ = T(5, 5) · S(2, 2) · R(90) · T(−2, −1) · P	24	75
M7	Snowman	Midpoint circle · P′ = T(4, −1) · S(1, 2) · R(−90) · P	26	83
M8	Clock	Bresenham line · P′ = T(2, 6) · S(−1, 1) · T(−2, −6) · P	28	90
M9	Sailboat	Midpoint ellipse · P′ = R(60) · S(2, 0.5) · T(−3, 2) · P	30	97
M10	Landscape	DDA line · P′ = T(1, 2) · R(30) · R(60) · T(−1, −2) · P	32	105

How to use this book

- Read Part 1 once — it has the steps of every question type.
- Solve one model in 1 hour without looking (Part 2), then correct it from Part 3.
- Use Part 4 to check the True/False and Complete parts quickly.

كل نموذج = ورقتين زي الامتحان بالظبط، والحل في جزء لوحده عشان متشوفهوش وانت بتحل.

CG / PART 1 / SOLVING GUIDE

Solving Guide

01

How to solve every question type in the exam — read once before the models

THEORY Question 1 — theory (all types)

Source: Lec 1–6 · SEC 1–7

1. **Define:** write the lecture sentence of the term — one or two lines.
2. **Compare / Differentiate:** a table with two columns and the same points in each row.
3. **True / False:** put ✓ or ✗; every false statement must be corrected with the right sentence.
4. **Complete:** the missing words exactly as in the lecture.
5. **Explain / List / Why:** numbered points, each one a lecture sentence.
6. **Calculate:** short problems (colors, LUT, frame buffer, window → viewport, aspect ratio, first pixels) — write the law, then the numbers.

```
Example from the reference → Define frame buffer → "A block of memory dedicated to graphics output which holds data representing all the pixels in a complete video frame."
```

الفكرة النظري بيتجاوب بجمل المحاضرة نفسها. والـ False مش بيكفي فيه ✗ لوحدها، لازم تكتب التصحيح.

W → V Window to viewport mapping

Source: Lec 3 · p5–6 · Graphics_Sec 4 · p1–2

1. Write the window (W.l, W.r, W.b, W.t) and the viewport (V.l, V.r, V.b, V.t).
2. Sx = (V.r − V.l) / (W.r − W.l) · Sy = (V.t − V.b) / (W.t − W.b)
3. XV = V.l + Sx (XW − W.l) · YV = V.b + Sy (YW − W.b)
4. If (XV, YV) is given and (XW, YW) is asked, use the same two equations and solve for XW and YW.

```
Example from the reference → Window (20, 80, 40, 80), viewport (30, 60, 40, 60), point (30, 80): Sx = Sy = 0.5 → XV = 35, YV = 60
```

الفكرة احسب Sx وSy الأول، وبعدين عوّض. لو مدّيك نقطة في الـ viewport، حل نفس المعادلتين بالعكس.

ASPECT Matching viewport (aspect ratio)

Source: Lec 3 · p24–26 · Graphics_Sec 4 · p2

1. R = aspect ratio of the world window; compare it with W/H of the screen window.
2. R > W/H → viewport width = W, height = W/R.
3. R < W/H → viewport height = H, width = H·R.
4. Answer as glViewport(0, 0, width, height).

```
Example from the reference → R = 1.6, W = 360, H = 200: W/H = 1.8 > R → height 200, width 320 → glViewport(0, 0, 320, 200)
```

الفكرة قارن R بـ W/H: الأكبر فيهم بيحدد مين اللي هياخد طول الشاشة كله.

COLOR Colors, LUT and frame buffer

Source: Lec 2 · p23, p35, p39 · Graphics_SEC 3 · p2

1. Gray-scale: N bits/pixel → 2^N gray levels.
2. Direct coding with n bits per primary color → $2^n \times 2^n \times 2^n$ colors.
3. LUT: entries = 2^(bits of the pixel value) · size = (entries × bits of each LUT entry) / 8 bytes.
4. Frame buffer size = width × height × bits per pixel / 8 bytes.

```
Example from the reference → 10-bit pixel, 24-bit LUT → 2¹⁰ = 1024 entries, (1024 × 24)/8 = 3072 bytes
```

الفكرة كله أُس 2: عدد الـ bits يتحول لعدد ألوان أو entries، والقسمة على 8 تحوّل الـ bits لـ bytes.

DDA DDA line algorithm

Source: Lec 4 · p19–23 · Graphics sec5 · p1

1. Δx = x_end − x0, Δy = y_end − y0, m = Δy/Δx, steps = max(|Δx|, |Δy|).
2. |m| ≤ 1: x changes by 1 each step, y changes by m.
3. |m| > 1: y changes by 1 each step, x changes by 1/m.
4. If the line goes from right to left (or down), subtract instead of add.
5. Round every value: round(a) = int(a + 0.5).

```
Example from the reference → (9, 8) → (2, 3): m ≤ 1, steps = 7 → x − 1, y − m: (8, 7.2) → (8, 7) …
```

الفكرة خطوة 1 على الـ coordinate اللي بيتغير أكتر، والتاني بيزيد بـ m أو 1/m وبعدين تقرّب.

BRES Bresenham line algorithm

Source: Lec 4 · p28–35 · Graphics sec5 · p2

1. Δx, Δy; if |Δy| > |Δx| swap the x points with the y points.
2. p0 = 2Δy − Δx.
3. $p_k < 0$ → next pixel (x + 1, y), $p_{k+1} = p_k + 2\Delta y$.
4. $p_k \geq 0$ → next pixel (x + 1, y + 1), $p_{k+1} = p_k + 2\Delta y - 2\Delta x$.
5. Repeat Δx times; write P, X, Y, SetPixel in a table.

```
Example from the reference → (20, 10) → (30, 18): p0 = 16 − 10 = 6 → (21, 11), then p − 4 or p + 16
```

الفكرة الـ decision parameter بيقولك تطلع لفوق ولا لأ: سالب يبقى y ثابت، موجب يبقى y + 1.

CIRCLE Midpoint circle algorithm

Source: Lec 5 · p10–14 · Graphics_Sec 6 · p1

1. Start at (0, r), P0 = 1 − r.
2. $P_k < 0$ → (x + 1, y), $P_{k+1} = P_k + 2x_{k+1} + 1$.
3. $P_k \geq 0$ → (x + 1, y − 1), $P_{k+1} = P_k + 2x_{k+1} + 1 - 2y_{k+1}$.
4. Stop when x ≥ y (one octant); the other seven octants come from symmetry.

```
Example from the reference → r = 10: P0 = −9 → (1, 10), P = −6 → (2, 10), P = −1 → (3, 10), P = 6 → (4, 9) …
```

الفكرة بنحسب octant واحد بس، وكل نقطة (x, y) بتدينا 8 نقط بالـ symmetry.

ELLIPSE Midpoint ellipse algorithm

Source: Lec 5 · p16–23 · Graphics_Sec 6 · p2

1. Region 1 starts at (0, r_y): $p1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2$. Increment x, calculate y.
2. $p1 \leq 0$ → (x + 1, y), $p1 + 2r_y^2 x_{k+1} + r_y^2$ · $p1 > 0$ → (x + 1, y − 1), $p1 + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2$.
3. Leave region 1 when $2r_y^2 x \geq 2r_x^2 y$.
4. Region 2 starts at the last point of region 1: $p2_0 = r_y^2(x_0 + \frac{1}{2})^2 + r_x^2(y_0 - 1)^2 - r_x^2 r_y^2$. Decrement y, calculate x.
5. $p2 \leq 0$ → (x + 1, y − 1), $p2 + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2$ · $p2 > 0$ → (x, y − 1), $p2 - 2r_x^2 y_{k+1} + r_x^2$. Stop at y = 0.

```
Example from the reference → r_x = 4, r_y = 3: p1₀ = 9 − 48 + 4 = −35 → (1, 3) …
```

الفكرة Region 1 بنزوّد x، وRegion 2 بنقلّل y. وبنحسب العمودين $2r_y^2 x$ و$2r_x^2 y$ في كل صف عشان نعرف إمتى نغيّر الـ region.

COMPOSITE

Deconstruct a composite transformation

Source: Lec 6 · p20–33 · Lec 1 · p22 · Graphics_Sec 7 · p3

1. Read from right to left: the matrix nearest to P is applied first.
2. T(−a, −b) … T(a, b) around an operation = do it around the point (a, b) (pivot / fixed point).
3. R(θ): positive θ → counter-clockwise, negative → clockwise, around the origin.
4. S(s_x, s_y): scale around the origin; a negative factor gives a mirror image.
5. Translations add, rotation angles add, scales multiply.

```
Example from the reference → P′ = T(3, 4)·R(40)·T(−3, −4)·P → rotate P by 40° around the point (3, 4)
```

الفكرة اقرا من اليمين للشمال، واكتب جملة لكل matrix، وفي الآخر جملة بتلخص اللي حصل للشكل كله.

PROGRAM

Writing the program (Question 3)

Source: Lec 3 · p10–11 · Lec 5 · p14

1. main: glutInit, glutInitDisplayMode, glutInitWindowSize, glutInitWindowPosition, glutCreateWindow, myInit(), glutDisplayFunc(myDisplay), glutMainLoop().
2. myInit: glClearColor, glMatrixMode(GL_PROJECTION), glLoadIdentity, gluOrtho2D(left, right, bottom, top).
3. myDisplay: glClear(GL_COLOR_BUFFER_BIT), then for every part: glColor3f → glBegin(mode) → glVertex2f … → glEnd(); finally glFlush().
4. Modes used in the answers: GL_POINTS, GL_LINES, GL_LINE_STRIP, GL_LINE_LOOP, GL_TRIANGLES, GL_QUADS, GL_POLYGON.
5. Circles: circleMidpoint(xc, yc, r, mode) from Lec 5 · with GL_POINTS you get the outline, with GL_LINES the circle is filled.

```
Example from the reference → Draw the background first and the small details last — what is drawn later appears on top.
```

الفكرة قسّم الصورة لأشكال بسيطة (مربعات ومثلثات ودوايرر)، وارسم كل شكل بلونه.

CG / PART 1 / THEORY COVERAGE MAP

Theory Coverage Map

Every Question 1 item of the 10 models, grouped by the lecture it comes from — use it to revise one lecture at a time

Lec 1 — 34 items

M2	T/F	Image processing is concerned with the creation of images.	p6
M6	Define	Image processing	p6
M1	Define	Computer graphics	p7
M7	Complete	We use … to model light and to perform simulations for animation, and	p7
M2	Complete	Geometric model describes …, while mathematical model describes ….	p9
M2	Compare	Interactive and non-interactive computer graphics	p11
M1	Define	Frame buffer	p12
M6	Explain	Explain the role of the CPU, the GPU and the frame buffer.	p12
M1	T/F	A valuator input primitive is scalar values (typically between 0 and 1	p13
M4	Complete	Locator input primitive is …; pick input primitive is ….	p13–14
M6	Define	Valuator	p13
M8	Explain	List the input devices and their input primitives.	p13–14
M1	Compare	Raster display and vector display	p15–16
M1	T/F	Enlarging a raster (bitmap) image keeps it sharp.	p15
M8	Define	Raster display	p15
M8	Complete	Vector graphics is known as ….	p15
M1	Define	Application model	p17
M4	Define	Graphics library (API)	p17
M7	T/F	The software level contains the application program, the application m	p17
M10	Compare	Hardware level and software level of interactive computer graphics	p17
M6	T/F	In 2D there is no need to simulate lights, cameras, etc.	p18
M9	Explain	Why is 2D learned before 3D?	p18
M6	T/F	The primary interface of OpenGL is in Java.	p19
M1	T/F	In display (physical) coordinates, Y increases up.	p20
M5	Compare	Cartesian coordinates and display (physical) coordinates	p20
M1	T/F	OpenGL uses normalized device coordinates: the coordinates of each ver	p21
M5	Complete	The region given by gluOrtho2D(xmin, xmax, ymin, ymax) is known as the	p21
M7	T/F	glutInitWindowPosition gives the location of the lower-left corner of	p21

M4	T/F	Positive angles rotate counter-clockwise about the origin.	p22
M1	T/F	Transformations can be composed, and their order does not matter.	p23
M1	Explain	Explain the stages of the OpenGL graphics pipeline.	p24
M9	Define	Clipping	p24
M9	Define	Rasterization	p24
M9	T/F	The vertex shader adds values for each pixel (coloring object shape).	p24

SEC 1 and 2 — 3 items

M7	Define	Output device	p1
M7	T/F	2-D vector output devices physically draw line segments (like a plotte	p1
M4	Define	Conceptual framework of computer graphics	p2

Lec 2 — 43 items

M1	Complete	From model to image: Model → … → View → … → Display.	p3
M4	Explain	Explain the steps of transforming coordinates from modeling to device coordinates.	p9
M3	Complete	Output primitives (elements of pictures) are …, …, … and ….	p10
M10	Define	Attributes of a graphic primitive	p10
M2	Complete	Polygon is a closed polyline where its … equal to its ….	p12
M3	Define	Polyline	p12
M4	T/F	In a polyline, each line is called a vertex.	p12
M7	Compare	Simple, complex, concave and convex polygons	p12
M9	Define	Polygon	p12
M2	T/F	A special case arises when a line segment shrinks to a single point, a	p13
M6	Complete	Attributes of lines and polylines are …, …, … and the way edges are bl	p14
M2	Complete	Text attributes are font face, …, size, style, spacing and ….	p15
M3	T/F	The attributes of a filled region are enclosing border, pattern and co	p17
M4	Define	Bitmap	p19
M7	Complete	In a raster image each numerical value of the 2D array represents the	p19
M3	T/F	Hand designed images are made by digitizing a photograph.	p20
M3	Explain	How are raster images created?	p20
M7	Define	Monochrome raster image	p22
M9	Compare	Monochrome, gray-scale and color raster images	p22–24
M1	Complete	In gray-scale images, N bit has … possible color values.	p23
M5	Calculate	How many gray levels are produced by 2, 4 and 8 bits/pixel?	p23
M6	T/F	4 bits/pixel produce 16 gray levels.	p23

M2	T/F	A true color image has a color depth of 16 bit.	p24
M2	Define	Pixel and scan line	p25
M4	Complete	The most familiar raster displays are flat panels, which include …, …,	p26
M4	Compare	Emissive and non-emissive displays	p27
M9	T/F	Emissive displays use optical effects to convert light from some other	p27
M3	T/F	When the GPU frame rate is higher than the monitor refresh rate, stutt	p28
M6	Compare	Frame rate and refresh rate	p28
M6	Complete	If the frame rate and the refresh rate are not equal, the … one will p	p28
M4	T/F	Nowadays the entire frame of pixel data is sent from the graphics card	p29
M10	Complete	In a raster scan system the … takes care of the actual display process	p30
M5	T/F	In a simple color frame buffer each bit plane drives an individual col	p33
M5	T/F	With 3 bit planes, one for each primary color, the frame buffer can sh	p33
M3	Compare	Direct coding and Look-Up Table (LUT)	p35–36
M8	Complete	A frame buffer that supports true color has … planes of memory, … plan	p35
M8	Calculate	A frame buffer supports 6 planes of memory (2 planes for each color).	p35
M10	Define	Color palette	p35
M10	T/F	In true color each DAC has eight input bits: 256 levels of each color,	p35
M10	Complete	After setPalette(39, 17, 25, 4), to make the pixel (479, 532) glow in	p37
M9	Calculate	If we use 8 bits pixel value in a 24-bit lookup table, find the LUT si	p38
M1	Calculate	If we use 10 bits pixel value in a 24-bit lookup table, how many entri	p39
M2	Calculate	If we use direct coding of RGB value with 3 bits per primary color, ho	p39

Graphics_SEC 3 — 7 items

M4	T/F	A concave polygon has all its diagonals inside the polygon.	p1
M10	T/F	NDC are the coordinates mapped to the specific resolution of the outpu	p1
M10	Explain	List the five stages of the graphics pipeline.	p1
M3	Calculate	If we use direct coding of RGB value with 5 bits per primary color, ho	p2
M4	Calculate	A frame buffer supports 12 planes of memory (4 planes for each color).	p2
M6	Calculate	System A: 1920 × 1080 display, 32 bits/pixel, no LUT. Find the frame b	p2
M10	Calculate	If we use a 10-bit pixel value in a 32-bit lookup table, find the entr	p2

Lec 3 — 10 items

M8	T/F	To ensure device independence, graphics systems convert object descrip	p2
M1	Complete	Window is … to be displayed, viewport is … to be displayed.	p3
M3	Define	Window to viewport mapping	p3

M8	T/F	The viewport is what to be displayed.	p3
M10	Calculate	Window (0, 2, 0, 1), viewport (40, 400, 60, 300). Find Sx and Sy.	p5
M1	Calculate	Window (W.l, W.r, W.b, W.t) = (20, 80, 40, 80), viewport (30, 60, 40,	p6
M6	Define	Extent (bounding box)	p21
M7	Explain	How can the application set the window automatically?	p21
M3	Calculate	Find the matching viewport for a world window with aspect ratio R = 1.	p26
M7	Calculate	Find the matching viewport for a window with aspect ratio R = 2 when t	p27

Graphics_Sec 4 — 1 items

M5	Calculate	Window (0, 10, 0, 5), viewport (50, 250, 100, 200). A viewport point i	p2

Lec 4 — 20 items

M3	Define	Vector	p3
M6	Explain	What are the three basic elements? Define each.	p3
M3	Complete	Each integer screen position references the … of a pixel area.	p5
M3	Complete	setPixel stores the current color setting into the frame buffer at int	p6
M1	Explain	How does a raster system display a straight line?	p7
M7	Define	Sampling	p8
M2	Explain	What are the trivial situations that do not need line-drawing algorithms?	p10
M5	Complete	In y = m·x + b: m = … and b = ….	p12
M7	Explain	Why does the simple approach (increment x, solve for y) fail for lines with slope greater than 1?	p13
M5	T/F	In DDA a line is sampled at unit intervals in one coordinate and the n	p15
M2	T/F	In DDA, if m > 1 we sample at unit x intervals.	p17
M4	Calculate	Using DDA, find the first three pixels of the line from (9, 8) to (2,	p23
M2	Explain	List the advantages and disadvantages of the DDA algorithm.	p24
M8	Compare	DDA and Bresenham line algorithms	p24–28
M10	T/F	In DDA the method of round-off is also time-consuming.	p24
M2	T/F	Bresenham's line algorithm uses only incremental integer calculations.	p25
M9	Complete	Bresenham's method is described for a line segment with a positive slo	p25
M8	Complete	In Bresenham, for xk+1 = xk + 1 we choose between … and ….	p27
M9	Calculate	Bresenham's line from (10, 0) to (20, 8): find the first four pixels a	p28–30
M2	Calculate	For Bresenham's line from (20, 10) to (30, 18), find p0 and the two up	p30

Lec 5 — 14 items

M10	Define	Circle	p3
M2	Define	Unit circle	p4
M10	T/F	Drawing a circle with $y = \sqrt{(r^2 - x^2)}$ gives uniform pixel spacing.	p5
M9	Complete	For a circle with center (xc, yc): x = xc + … and y = yc + ….	p8
M4	T/F	In the midpoint circle algorithm, if P > 0 the midpoint is inside t	p9, p12
M9	T/F	In the midpoint circle algorithm $P = x^2 + y^2 - R^2$; if P = 0 the point	p9
M7	T/F	The midpoint circle algorithm must calculate the points of all eight o	p10, p13
M6	Calculate	Midpoint circle with r = 10: find P0 and the first three points.	p12
M4	Complete	In region 1 of the ellipse we increment … and calculate ….	p17
M7	Calculate	For an ellipse with rx = 8 and ry = 6 find the initial point and $p1_0$.	p22
M2	Define	Aliasing	p24
M5	T/F	There is no limit to how big we can make the frame buffer and still ke	p24
M5	Define	Super-sampling	p25
M7	Complete	Supersampling gives more weight to subpixels near the … of a pixel are	p25

Lec 6 — 28 items

M3	Explain	List the geometric transformation functions.	p2
M3	T/F	Linear transformations preserve co-linearity and ratios of distances.	p3
M5	Complete	The midpoint of a line segment remains the … after a linear transforma	p3
M8	Define	Linear transformation	p3, p5
M3	T/F	Scaling preserves the angles between lines for any sx, sy.	p6
M8	T/F	In scaling, the object must be centered on the origin, or it will be s	p6
M4	Explain	Explain the three types of 2D reflection.	p7
M5	Define	Reflection	p7
M5	Explain	Derive the 2D rotation about the origin.	p8–10
M8	Calculate	Rotate the point B(2, 1) by 90° about the origin.	p10, p34
M5	Define	Shear (skewing)	p11
M7	T/F	Shear in y: $x' = x + shy \cdot y$.	p11
M9	Complete	Shear (skewing) deforms shape like ….	p11
M8	Define	Translation	p12
M6	T/F	In Cartesian form translation is a multiplication $P' = T \cdot P$.	p13–15
M5	Explain	Why do we use homogeneous coordinates?	p15
M6	T/F	Two translations combine by multiplying their values.	p20–22
M10	T/F	The inverse translation $T^{-1}(tx, ty) = T(-tx, -ty)$.	p22
M8	T/F	$S(sx1, sy1) \cdot S(sx2, sy2) = S(sx1 + sx2, sy1 + sy2)$.	p24

M9	T/F	The inverse of the scaling S(sx, sy) is S(−sx, −sy).	p24
M5	T/F	The inverse rotation $R^{-1}(\theta) = R^{T}(\theta)$.	p26
M6	Complete	Rotation property: R(θ) · R(φ) = ….	p26
M9	T/F	R(θ) · R(φ) = R(φ) · R(θ).	p26
M8	Explain	What is a 2D composite transformation and when is it used?	p29
M10	Complete	Composite transformations are used to rotate or scale an object w.r.t.	p29
M10	Explain	Write the steps to rotate an object around a pivot point.	p30
M8	T/F	Rotation in 3D needs to specify which axis the rotation is about.	p40
M9	Explain	In 3D, how do we scale an object w.r.t. a fixed position (x_f, y_f, z_f)?	p42

PART 2 · EXAM PAPERS

Exam Papers

Ten models in the layout of the real mid-term: Question 1 (theory: define, compare, true/false, complete, explain), Question 2 (algorithm + composite transformation), Question 3 (program). One hour each.

حل كل نموذج في ساعة من غير ما تفتح الحل.

CG / EXAM PAPER / MODEL 01

Practice Model 1

M1

Exam paper · answers in Part 3, page 35

Subject: (CMPn322) Computer graphics
Academic Year: 2025-2026
Semester: Fall
Branch: Computer
Time: 1 hour
Academic Level: Senior 1

Questions FOR THE MID TERM WRITTEN EXAM — Practice Model 1

Number of Pages: 2
Number of Questions: 3
Attempt all questions

Question1 , [LO's 1,2,3,4] (5 Points uniformly distributed)

Answer the following:

a. `Define` **Frame buffer** · **Computer graphics** · **Application model**.

b. `Compare` Differentiate between **Raster display and vector display**.

c. `True / False` Put (✓) or (✗), and correct the false statements:

1. Transformations can be composed, and their order does not matter. ( )
2. A valuator input primitive is scalar values (typically between 0 and 1). ( )
3. In display (physical) coordinates, Y increases up. ( )
4. OpenGL uses normalized device coordinates: the coordinates of each vertex should be between −1.0 and 1.0. ( )
5. Enlarging a raster (bitmap) image keeps it sharp. ( )

d. `Complete`

1. Window is _____ to be displayed, viewport is _____ to be displayed.
2. In gray-scale images, N bit has _____ possible color values.
3. From model to image: Model → _____ → View → _____ → Display.

e. `Explain`

1. Explain the stages of the OpenGL graphics pipeline.
2. How does a raster system display a straight line?

f. `Calculate`

1. If we use 10 bits pixel value in a 24-bit lookup table, how many entries does the LUT have and what is its size in bytes?
2. Window (W.l, W.r, W.b, W.t) = (20, 80, 40, 80), viewport (30, 60, 40, 60). Find (XV, YV) for (XW, YW) = (30, 80).

Practice Model 1 — Question 2 & Question 3

Question2 [LO's 2, 3] (10 Points)

1) Determine pixel positions along the ellipse path in the first quadrant using midpoint ellipse algorithm. Given: input ellipse parameter $r_x = 6$ and $r_y = 4$

- $p1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2$
- $p1_{k+1} = p1_k + 2r_y^2 x_{k+1} + r_y^2 \ (p1_k \le 0) \quad | \quad p1_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2 \ (p1_k > 0)$
- $p2_0 = r_y^2 (x_0 + \frac{1}{2})^2 + r_x^2 (y_0 - 1)^2 - r_x^2 r_y^2$
- $p2_{k+1} = p2_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2 \ (p2_k \le 0) \quad | \quad p2_k - 2r_x^2 y_{k+1} + r_x^2 \ (p2_k > 0)$

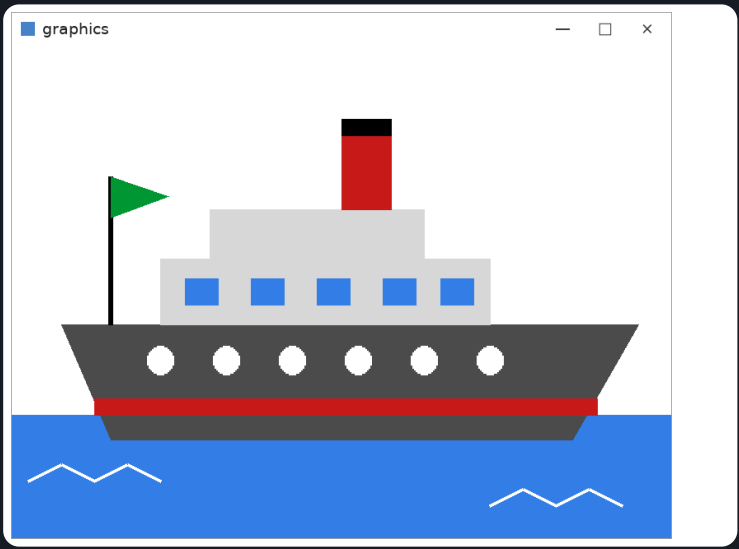
2) A programmer applies the following composite transformation to every vertex of a 2D object:

$$P' = T(6, 3) \cdot R(45) \cdot S(3, 1) \cdot T(-6, -3) \cdot P$$

Deconstruct the Transformation: Break down the sequence of operations. Describe what happens to the object step-by-step. (5 Points)

Question 3 [LO's 3,4] (5 Points)

Write a program to draw the following fig



CG / EXAM PAPER / MODEL 02

Practice Model 2

M2

Exam paper · answers in Part 3, page 44

Subject: (CMPn322) Computer graphics
Academic Year: 2025-2026
Semester: Fall
Branch: Computer
Time: 1 hour
Academic Level: Senior 1

Questions FOR THE MID TERM WRITTEN EXAM — Practice Model 2

Number of Pages: 2
Number of Questions: 3
Attempt all questions

Question1 , [LO's 1,2,3,4] (5 Points uniformly distributed)

Answer the following:

a. `Define` **Aliasing** · **Pixel and scan line** · **Unit circle**.

b. `Compare` Differentiate between **Interactive and non-interactive computer graphics**.

c. `True / False` Put (✓) or (✗), and correct the false statements:

1. Image processing is concerned with the creation of images. ( )
2. Bresenham's line algorithm uses only incremental integer calculations. ( )
3. A true color image has a color depth of 16 bit. ( )
4. A special case arises when a line segment shrinks to a single point, and it is drawn as a "dot". ( )
5. In DDA, if m > 1 we sample at unit x intervals. ( )

d. `Complete`

1. Geometric model describes _____, while mathematical model describes _____.
2. Polygon is a closed polyline where its _____ equal to its _____.
3. Text attributes are font face, _____, size, style, spacing and _____.

e. `Explain`

1. List the advantages and disadvantages of the DDA algorithm.
2. What are the trivial situations that do not need line-drawing algorithms?

f. `Calculate`

1. If we use direct coding of RGB value with 3 bits per primary color, how many possible colors do we have for each pixel?
2. For Bresenham's line from (20, 10) to (30, 18), find p0 and the two update values.

Practice Model 2 — Question 2 & Question 3

Question2 [LO's 2, 3] (10 Points)

1) Given a circle radius r = 14, determine positions along the circle octant in the first quadrant from x = 0 to x = y using the midpoint circle algorithm.

- Initial point (0, r) & $P0 = 1 - r$
- $p_k < 0$: $y_{k+1} = y_k$, $P_{k+1} = P_k + 2x_{k+1} + 1$
- $p_k \geq 0$: $y_{k+1} = y_k - 1$, $P_{k+1} = P_k + 2x_{k+1} + 1 - 2y_{k+1}$

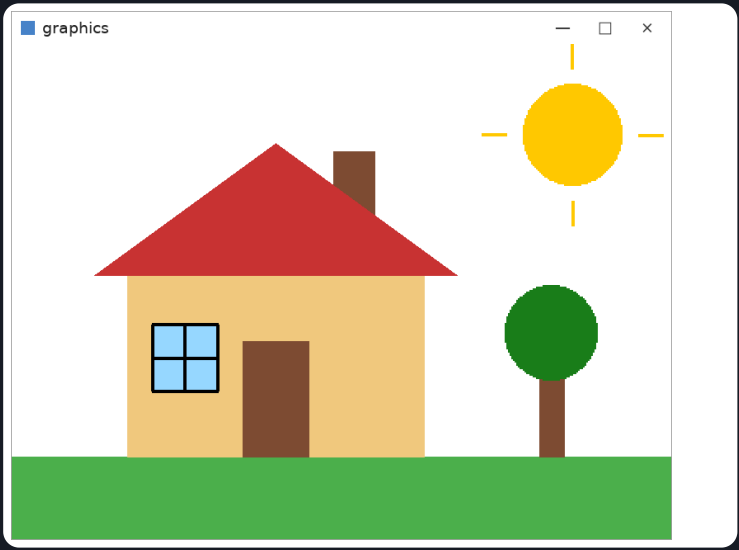
2) A programmer applies the following composite transformation to every vertex of a 2D object:

$$P' = R(30) \cdot T(5, -2) \cdot S(0.5, 0.5) \cdot P$$

Deconstruct the Transformation: Break down the sequence of operations. Describe what happens to the object step-by-step. (5 Points)

Question 3 [LO's 3,4] (5 Points)

Write a program to draw the following fig



CG / EXAM PAPER / MODEL 03

Practice Model 3

M3

Exam paper · answers in Part 3, page 52

Subject: (CMPn322) Computer graphics
Academic Year: 2025-2026
Semester: Fall
Branch: Computer
Time: 1 hour
Academic Level: Senior 1

Questions FOR THE MID TERM WRITTEN EXAM — Practice Model 3

Number of Pages: 2
Number of Questions: 3
Attempt all questions

Question1 , [LO's 1,2,3,4] (5 Points uniformly distributed)

Answer the following:

a. `Define` **Polyline** · **Window to viewport mapping** · **Vector**.

b. `Compare` Differentiate between **Direct coding and Look-Up Table (LUT)**.

c. `True / False` Put (✓) or (✗), and correct the false statements:

1. When the GPU frame rate is higher than the monitor refresh rate, stuttering happens. ( )
2. Linear transformations preserve co-linearity and ratios of distances. ( )
3. Scaling preserves the angles between lines for any s_x, s_y. ( )
4. Hand designed images are made by digitizing a photograph. ( )
5. The attributes of a filled region are enclosing border, pattern and color. ( )

d. `Complete`

1. Output primitives (elements of pictures) are _____, _____, _____ and _____.
2. Each integer screen position references the _____ of a pixel area.
3. setPixel stores the current color setting into the frame buffer at integer position (x, y); _____ retrieves the current frame-buffer setting for a pixel location.

e. `Explain`

1. How are raster images created?
2. List the geometric transformation functions.

f. `Calculate`

1. If we use direct coding of RGB value with 5 bits per primary color, how many possible colors do we have?
2. Find the matching viewport for a world window with aspect ratio R = 1.6 when the screen window has width 360 and height 200.

Practice Model 3 — Question 2 & Question 3

Question2 [LO's 2, 3] (10 Points)

1) Use Bresenham's line drawing algorithm to digitize the line with endpoints (15, 20) and (25, 26). Show the decision parameter at each step and plot the pixels.

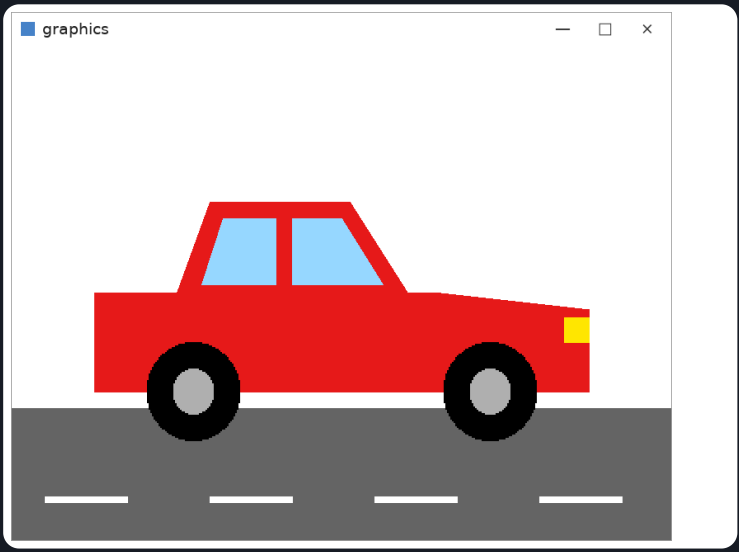
2) A programmer applies the following composite transformation to every vertex of a 2D object:

$$P' = T(-2, 4) \cdot S(1, -1) \cdot R(-60) \cdot P$$

Deconstruct the Transformation: Break down the sequence of operations. Describe what happens to the object step-by-step. (5 Points)

Question 3 [LO's 3,4] (5 Points)

Write a program to draw the following fig



CG / EXAM PAPER / MODEL 04

Practice Model 4

M4

Exam paper · answers in Part 3, page 59

Subject: (CMPn322) Computer graphics
Academic Year: 2025-2026
Semester: Fall
Branch: Computer
Time: 1 hour
Academic Level: Senior 1

Questions FOR THE MID TERM WRITTEN EXAM — Practice Model 4

Number of Pages: 2
Number of Questions: 3
Attempt all questions

Question1 , [LO's 1,2,3,4] (5 Points uniformly distributed)

Answer the following:

a. `Define` **Graphics library (API)** · **Bitmap** · **Conceptual framework of computer graphics**.

b. `Compare` Differentiate between **Emissive and non-emissive displays**.

c. `True / False` Put (✓) or (✗), and correct the false statements:

1. A concave polygon has all its diagonals inside the polygon. ( )
2. Positive angles rotate counter-clockwise about the origin. ( )
3. In the midpoint circle algorithm, if P > 0 the midpoint is inside the circle. ( )
4. Nowadays the entire frame of pixel data is sent from the graphics card to the monitor digitally (HDMI, DisplayPort), row by row. ( )
5. In a polyline, each line is called a vertex. ( )

d. `Complete`

1. Locator input primitive is _____; pick input primitive is _____.
2. In region 1 of the ellipse we increment _____ and calculate _____.
3. The most familiar raster displays are flat panels, which include _____, _____, digital signage and automotive infotainment screens.

e. `Explain`

1. Explain the steps of transforming coordinates from modeling to device coordinates.
2. Explain the three types of 2D reflection.

f. `Calculate`

1. A frame buffer supports 12 planes of memory (4 planes for each color). Calculate the total number of colors.
2. Using DDA, find the first three pixels of the line from (9, 8) to (2, 3).

Practice Model 4 — Question 2 & Question 3

Question2 [LO's 2, 3] (10 Points)

1) Determine pixel positions along the ellipse path in the first quadrant using midpoint ellipse algorithm. Given: input ellipse parameter $r_x = 10$ and $r_y = 6$

- $p1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2$
- $p1_{k+1} = p1_k + 2r_y^2 x_{k+1} + r_y^2 \ (p1_k \le 0) \quad | \quad p1_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2 \ (p1_k > 0)$
- $p2_0 = r_y^2 (x_0 + \frac{1}{2})^2 + r_x^2 (y_0 - 1)^2 - r_x^2 r_y^2$
- $p2_{k+1} = p2_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2 \ (p2_k \le 0) \quad | \quad p2_k - 2r_x^2 y_{k+1} + r_x^2 \ (p2_k > 0)$

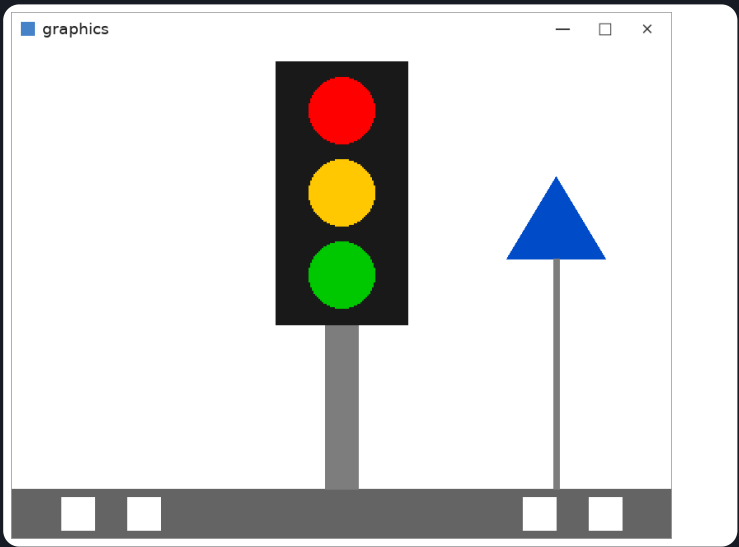
2) A programmer applies the following composite transformation to every vertex of a 2D object:

$$P' = T(3, 3) \cdot R(90) \cdot T(-3, -3) \cdot T(4, 0) \cdot P$$

Deconstruct the Transformation: Break down the sequence of operations. Describe what happens to the object step-by-step. (5 Points)

Question 3 [LO's 3,4] (5 Points)

Write a program to draw the following fig



CG / EXAM PAPER / MODEL 05

Practice Model 5

M5

Exam paper · answers in Part 3, page 67

Subject: (CMPn322) Computer graphics
Academic Year: 2025-2026
Semester: Fall
Branch: Computer
Time: 1 hour
Academic Level: Senior 1

Questions FOR THE MID TERM WRITTEN EXAM — Practice Model 5

Number of Pages: 2
Number of Questions: 3
Attempt all questions

Question1 , [LO's 1,2,3,4] (5 Points uniformly distributed)

Answer the following:

a. `Define` **Super-sampling** · **Reflection** · **Shear (skewing)**.

b. `Compare` Differentiate between **Cartesian coordinates and display (physical) coordinates**.

c. `True / False` Put (✓) or (✗), and correct the false statements:

1. In DDA a line is sampled at unit intervals in one coordinate and the nearest integer values are found for the other. ( )
2. There is no limit to how big we can make the frame buffer and still keep the refresh rate at 60 frames per second. ( )
3. In a simple color frame buffer each bit plane drives an individual color gun. ( )
4. With 3 bit planes, one for each primary color, the frame buffer can show 8 colors. ( )
5. The inverse rotation $R^{-1}(\theta) = R^{T}(\theta)$. ( )

d. `Complete`

1. The region given by gluOrtho2D(xmin, xmax, ymin, ymax) is known as the _____.
2. The midpoint of a line segment remains the _____ after a linear transformation.
3. In y = m·x + b: m = _____ and b = _____.

e. `Explain`

1. Why do we use homogeneous coordinates?
2. Derive the 2D rotation about the origin.

f. `Calculate`

1. How many gray levels are produced by 2, 4 and 8 bits/pixel?
2. Window (0, 10, 0, 5), viewport (50, 250, 100, 200). A viewport point is (150, 150); find (XW, YW).

Practice Model 5 — Question 2 & Question 3

Question2 [LO's 2, 3] (10 Points)

1) Use the DDA line drawing algorithm to determine the pixel positions along the line from (5, 4) to (12, 13) and plot them.

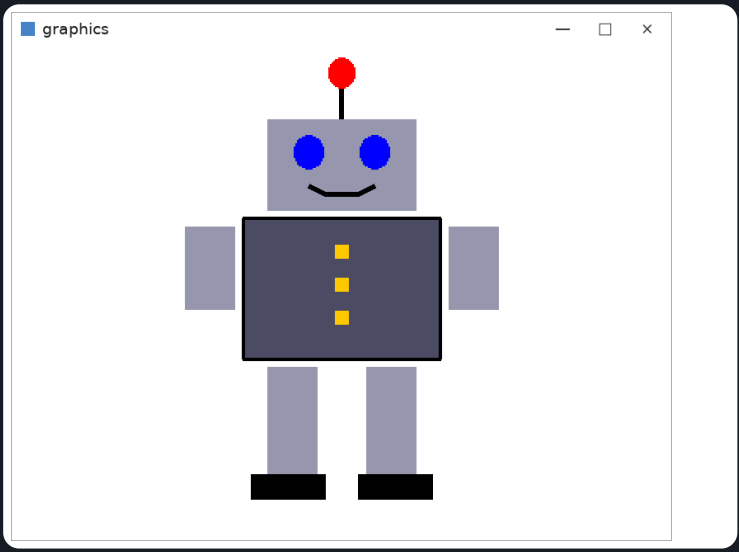
2) A programmer applies the following composite transformation to every vertex of a 2D object:

$$P' = S(2, 2) \cdot T(1, 1) \cdot R(180) \cdot P$$

Deconstruct the Transformation: Break down the sequence of operations. Describe what happens to the object step-by-step. (5 Points)

Question 3 [LO's 3,4] (5 Points)

Write a program to draw the following fig



CG / EXAM PAPER / MODEL 06

Practice Model 6

M6

Exam paper · answers in Part 3, page 75

Subject: (CMPn322) Computer graphics
Academic Year: 2025-2026
Semester: Fall
Branch: Computer
Time: 1 hour
Academic Level: Senior 1

Questions FOR THE MID TERM WRITTEN EXAM — Practice Model 6

Number of Pages: 2
Number of Questions: 3
Attempt all questions

Question1 , [LO's 1,2,3,4] (5 Points uniformly distributed)

Answer the following:

a. `Define` **Image processing** · **Extent (bounding box)** · **Valuator**.

b. `Compare` Differentiate between **Frame rate and refresh rate**.

c. `True / False` Put (✓) or (✗), and correct the false statements:

1. The primary interface of OpenGL is in Java. ( )
2. 4 bits/pixel produce 16 gray levels. ( )
3. Two translations combine by multiplying their values. ( )
4. In 2D there is no need to simulate lights, cameras, etc. ( )
5. In Cartesian form translation is a multiplication $P' = T \cdot P$. ( )

d. `Complete`

1. Attributes of lines and polylines are _____, _____, _____ and the way edges are blended.
2. Rotation property: $R(\theta) \cdot R(\varphi) =$ _____.
3. If the frame rate and the refresh rate are not equal, the _____ one will put a cap on the higher one.

e. `Explain`

1. Explain the role of the CPU, the GPU and the frame buffer.
2. What are the three basic elements? Define each.

f. `Calculate`

1. System A: 1920 × 1080 display, 32 bits/pixel, no LUT. Find the frame buffer memory.
2. Midpoint circle with r = 10: find P0 and the first three points.

Practice Model 6 — Question 2 & Question 3

Question2 [LO's 2, 3] (10 Points)

1) Determine pixel positions along the ellipse path in the first quadrant using midpoint ellipse algorithm. Given: input ellipse parameter $r_x = 8$ and $r_y = 4$

- $p1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2$
- $p1_{k+1} = p1_k + 2r_y^2 x_{k+1} + r_y^2 \ (p1_k \le 0) \quad | \quad p1_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2 \ (p1_k > 0)$
- $p2_0 = r_y^2 (x_0 + \frac{1}{2})^2 + r_x^2 (y_0 - 1)^2 - r_x^2 r_y^2$
- $p2_{k+1} = p2_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2 \ (p2_k \le 0) \quad | \quad p2_k - 2r_x^2 y_{k+1} + r_x^2 \ (p2_k > 0)$

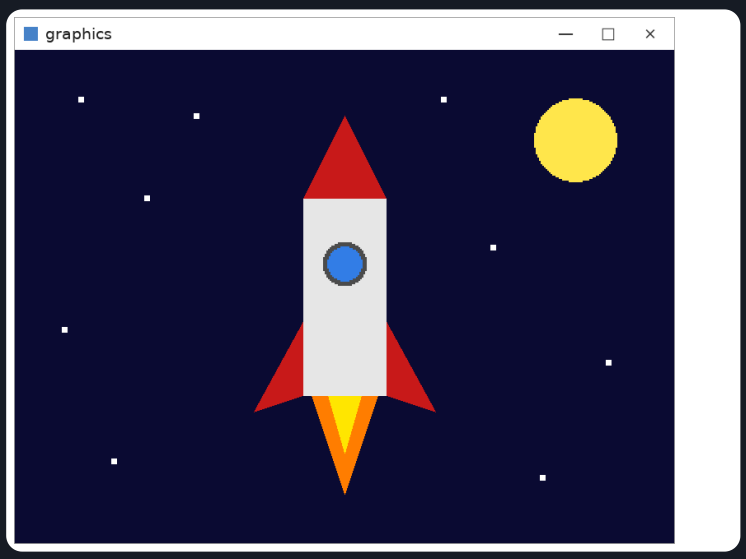
2) A programmer applies the following composite transformation to every vertex of a 2D object:

$$P' = T(5, 5) \cdot S(2, 2) \cdot R(90) \cdot T(-2, -1) \cdot P$$

Deconstruct the Transformation: Break down the sequence of operations. Describe what happens to the object step-by-step. (5 Points)

Question 3 [LO's 3,4] (5 Points)

Write a program to draw the following fig



CG / EXAM PAPER / MODEL 07

Practice Model 7

M7

Exam paper · answers in Part 3, page 83

Subject: (CMPn322) Computer graphics
Academic Year: 2025-2026
Semester: Fall
Branch: Computer
Time: 1 hour
Academic Level: Senior 1

Questions FOR THE MID TERM WRITTEN EXAM — Practice Model 7

Number of Pages: 2
Number of Questions: 3
Attempt all questions

Question1 , [LO's 1,2,3,4] (5 Points uniformly distributed)

Answer the following:

a. `Define` **Sampling** · **Monochrome raster image** · **Output device**.

b. `Compare` Differentiate between **Simple, complex, concave and convex polygons**.

c. `True / False` Put (✓) or (✗), and correct the false statements:

1. The midpoint circle algorithm must calculate the points of all eight octants. ( )
2. The software level contains the application program, the application model and the graphics library. ( )
3. glutInitWindowPosition gives the location of the lower-left corner of the display window. ( )
4. 2-D vector output devices physically draw line segments (like a plotter). ( )
5. Shear in y: $x' = x + sh_y \cdot y$. ( )

d. `Complete`

1. We use _____ to model light and to perform simulations for animation, and _____ to describe shape and motion.
2. Supersampling gives more weight to subpixels near the _____ of a pixel area.
3. In a raster image each numerical value of the 2D array represents the _____ stored there.

e. `Explain`

1. Why does the simple approach (increment x, solve for y) fail for lines with slope greater than 1?
2. How can the application set the window automatically?

f. `Calculate`

1. Find the matching viewport for a window with aspect ratio R = 2 when the screen window is 360 × 200.
2. For an ellipse with $r_x = 8$ and $r_y = 6$ find the initial point and $p1_o$.

Practice Model 7 — Question 2 & Question 3

Question2 [LO's 2, 3] (10 Points)

1) Given a circle radius r = 11, determine positions along the circle octant in the first quadrant from x = 0 to x = y using the midpoint circle algorithm.

- Initial point (0, r) & $P0 = 1 - r$
- $p_k < 0$: $y_{k+1} = y_k$, $P_{k+1} = P_k + 2x_{k+1} + 1$
- $p_k \geq 0$: $y_{k+1} = y_k - 1$, $P_{k+1} = P_k + 2x_{k+1} + 1 - 2y_{k+1}$

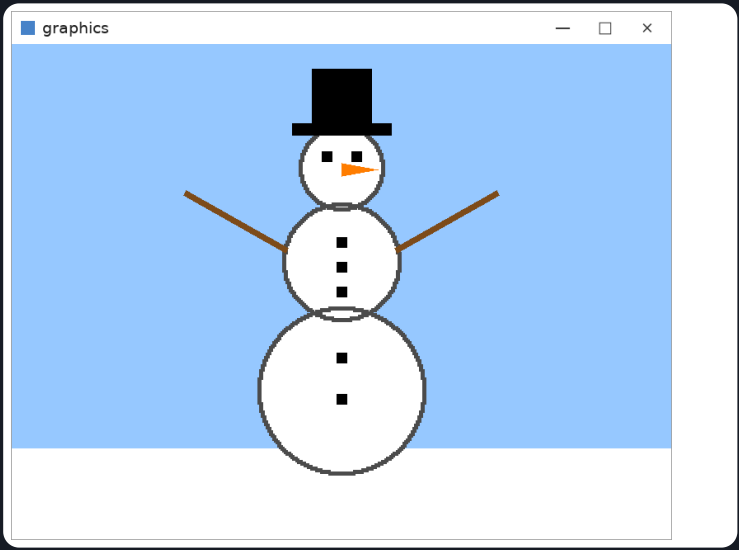
2) A programmer applies the following composite transformation to every vertex of a 2D object:

$$P' = T(4, -1) \cdot S(1, 2) \cdot R(-90) \cdot P$$

Deconstruct the Transformation: Break down the sequence of operations. Describe what happens to the object step-by-step. (5 Points)

Question 3 [LO's 3,4] (5 Points)

Write a program to draw the following fig



CG / EXAM PAPER / MODEL 08

Practice Model 8

M8

Exam paper · answers in Part 3, page 90

Subject: (CMPn322) Computer graphics
Academic Year: 2025-2026
Semester: Fall
Branch: Computer
Time: 1 hour
Academic Level: Senior 1

Questions FOR THE MID TERM WRITTEN EXAM — Practice Model 8

Number of Pages: 2
Number of Questions: 3
Attempt all questions

Question1 , [LO's 1,2,3,4] (5 Points uniformly distributed)

Answer the following:

a. `Define` **Linear transformation** · **Raster display** · **Translation**.

b. `Compare` Differentiate between **DDA and Bresenham line algorithms**.

c. `True / False` Put (✓) or (✗), and correct the false statements:

1. The viewport is what to be displayed. ( )
2. To ensure device independence, graphics systems convert object descriptions into normalized coordinates. ( )
3. In scaling, the object must be centered on the origin, or it will be shifted. ( )
4. Rotation in 3D needs to specify which axis the rotation is about. ( )
5. $S(s_{x1}, s_{y1}) \cdot S(s_{x2}, s_{y2}) = S(s_{x1} + s_{x2}, s_{y1} + s_{y2})$. ( )

d. `Complete`

1. Vector graphics is known as _____.
2. A frame buffer that supports true color has _____ planes of memory, _____ planes for each color.
3. In Bresenham, for $x_{k+1} = x_k + 1$ we choose between _____ and _____.

e. `Explain`

1. List the input devices and their input primitives.
2. What is a 2D composite transformation and when is it used?

f. `Calculate`

1. A frame buffer supports 6 planes of memory (2 planes for each color). How many colors are in the palette?
2. Rotate the point B(2, 1) by 90° about the origin.

Practice Model 8 — Question 2 & Question 3

Question2 [LO's 2, 3] (10 Points)

1) Use Bresenham's line drawing algorithm to digitize the line with endpoints (30, 18) and (42, 25). Show the decision parameter at each step and plot the pixels.

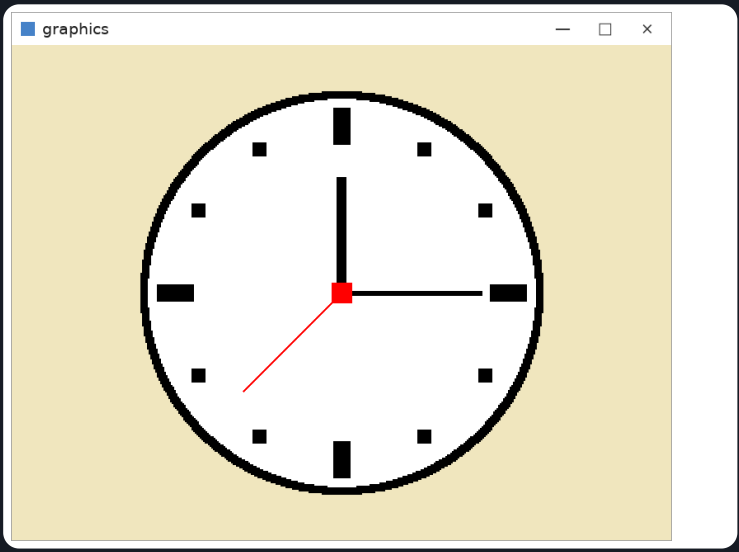
2) A programmer applies the following composite transformation to every vertex of a 2D object:

$$P' = T(2, 6) \cdot S(-1, 1) \cdot T(-2, -6) \cdot P$$

Deconstruct the Transformation: Break down the sequence of operations. Describe what happens to the object step-by-step. (5 Points)

Question 3 [LO's 3,4] (5 Points)

Write a program to draw the following fig



CG / EXAM PAPER / MODEL 09

Practice Model 9

M9

Exam paper · answers in Part 3, page 97

Subject: (CMPn322) Computer graphics
Academic Year: 2025-2026
Semester: Fall
Branch: Computer
Time: 1 hour
Academic Level: Senior 1

Questions FOR THE MID TERM WRITTEN EXAM — Practice Model 9

Number of Pages: 2
Number of Questions: 3
Attempt all questions

Question1 , [LO's 1,2,3,4] (5 Points uniformly distributed)

Answer the following:

a. `Define` **Clipping** · **Rasterization** · **Polygon**.

b. `Compare` Differentiate between **Monochrome, gray-scale and color raster images**.

c. `True / False` Put (✓) or (✗), and correct the false statements:

1. $R(\theta) \cdot R(\varphi) = R(\varphi) \cdot R(\theta)$. ( )
2. The inverse of the scaling $S(s_x, s_y)$ is $S(-s_x, -s_y)$. ( )
3. Emissive displays use optical effects to convert light from some other source. ( )
4. The vertex shader adds values for each pixel (coloring object shape). ( )
5. In the midpoint circle algorithm $P = x^2 + y^2 - R^2$; if $P = 0$ the point is on the circle. ( )

d. `Complete`

1. Shear (skewing) deforms shape like _____.
2. Bresenham's method is described for a line segment with a positive slope less than _____.
3. For a circle with center (x_c, y_c): $x = x_c +$ _____ and $y = y_c +$ _____.

e. `Explain`

1. Why is 2D learned before 3D?
2. In 3D, how do we scale an object w.r.t. a fixed position (x_f, y_f, z_f)?

f. `Calculate`

1. If we use 8 bits pixel value in a 24-bit lookup table, find the LUT size.
2. Bresenham's line from (10, 0) to (20, 8): find the first four pixels after (10, 0).

Practice Model 9 — Question 2 & Question 3

Question2 [LO's 2, 3] (10 Points)

1) Determine pixel positions along the ellipse path in the first quadrant using midpoint ellipse algorithm. Given: input ellipse parameter $r_x = 6$ and $r_y = 8$

- $p1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2$
- $p1_{k+1} = p1_k + 2r_y^2 x_{k+1} + r_y^2 \ (p1_k \le 0) \quad | \quad p1_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2 \ (p1_k > 0)$
- $p2_0 = r_y^2 (x_0 + \frac{1}{2})^2 + r_x^2 (y_0 - 1)^2 - r_x^2 r_y^2$
- $p2_{k+1} = p2_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2 \ (p2_k \le 0) \quad | \quad p2_k - 2r_x^2 y_{k+1} + r_x^2 \ (p2_k > 0)$

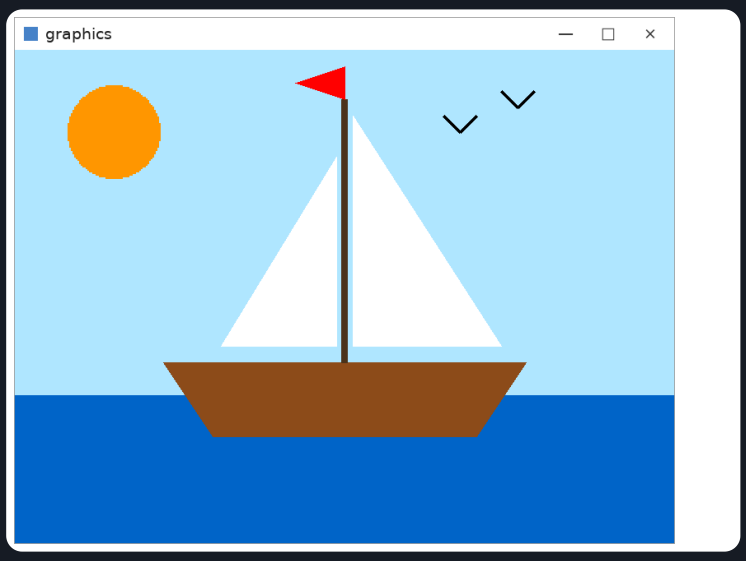
2) A programmer applies the following composite transformation to every vertex of a 2D object:

$$P' = R(60) \cdot S(2, 0.5) \cdot T(-3, 2) \cdot P$$

Deconstruct the Transformation: Break down the sequence of operations. Describe what happens to the object step-by-step. (5 Points)

Question 3 [LO's 3,4] (5 Points)

Write a program to draw the following fig



CG / EXAM PAPER / MODEL 10

Practice Model 10

M10

Exam paper · answers in Part 3, page 105

Subject: (CMPn322) Computer graphics
Academic Year: 2025-2026
Semester: Fall
Branch: Computer
Time: 1 hour
Academic Level: Senior 1

Questions FOR THE MID TERM WRITTEN EXAM — Practice Model 10

Number of Pages: 2
Number of Questions: 3
Attempt all questions

Question1 , [LO's 1,2,3,4] (5 Points uniformly distributed)

Answer the following:

a. `Define` **Circle** · **Attributes of a graphic primitive** · **Color palette**.

b. `Compare` Differentiate between **Hardware level and software level of interactive computer graphics**.

c. `True / False` Put (✓) or (✗), and correct the false statements:

1. The inverse translation $T^{-1}(t_x, t_y) = T(-t_x, -t_y)$. ( )
2. Drawing a circle with $y = \sqrt{(r^2 - x^2)}$ gives uniform pixel spacing. ( )
3. In DDA the method of round-off is also time-consuming. ( )
4. In true color each DAC has eight input bits: 256 levels of each color, 16 million colors in total. ( )
5. NDC are the coordinates mapped to the specific resolution of the output device (e.g. 1024 × 768). ( )

d. `Complete`

1. Composite transformations are used to rotate or scale an object w.r.t. a point that is not the _____.
2. In a raster scan system the _____ takes care of the actual display process.
3. After setPalette(39, 17, 25, 4), to make the pixel (479, 532) glow in this color, the _____ is stored in the frame buffer.

e. `Explain`

1. Write the steps to rotate an object around a pivot point.
2. List the five stages of the graphics pipeline.

f. `Calculate`

1. If we use a 10-bit pixel value in a 32-bit lookup table, find the entries and the size in bytes.
2. Window (0, 2, 0, 1), viewport (40, 400, 60, 300). Find Sx and Sy.

Practice Model 10 — Question 2 & Question 3

Question2 [LO's 2, 3] (10 Points)

1) Use the DDA line drawing algorithm to determine the pixel positions along the line from (14, 12) to (6, 7) and plot them.

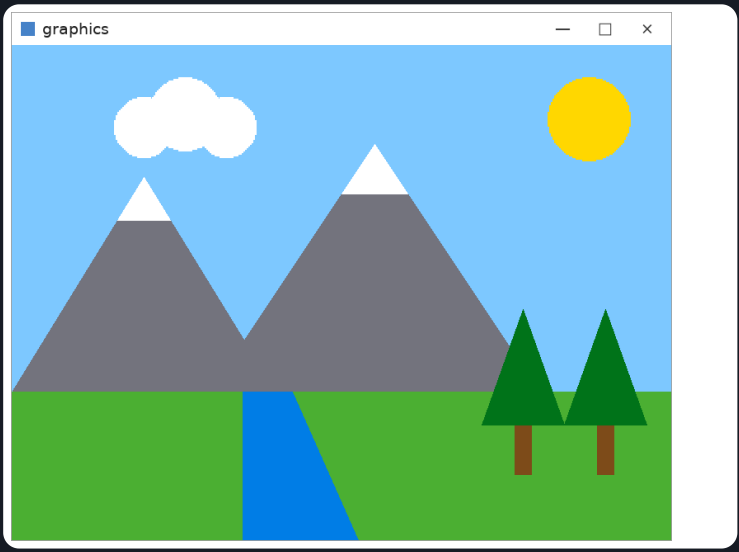
2) A programmer applies the following composite transformation to every vertex of a 2D object:

$$P' = T(1, 2) \cdot R(30) \cdot R(60) \cdot T(-1, -2) \cdot P$$

Deconstruct the Transformation: Break down the sequence of operations. Describe what happens to the object step-by-step. (5 Points)

Question 3 [LO's 3,4] (5 Points)

Write a program to draw the following fig



PART 3 · MODEL ANSWERS

Model Answers

Full answers for the ten models: Question 1 with the lecture sentences and source, Question 2 with "how to solve" steps and tables, Question 3 with the program and the coordinate grid.

في أي حاجة مش واضحة Solving Guide وارجع للـ سؤال سؤال، صحّح نفسك.

CG / MODEL ANSWER / MODEL 01

Model Answer — Model 1

M1

Exam paper on page 14 · each answer shows the lecture page of its method

Q1

Source: the page is next to each answer

Question 1 — theory

a · Define

Frame buffer: A block of memory dedicated to graphics output which holds data representing all the pixels in a complete video frame. Lec 1 · p12

Computer graphics: Computer graphics means creation, storage and manipulation of models and images. Lec 1 · p7

Application model: It captures all the data or objects to be pictured on the screen, that stored in application database. Lec 1 · p17

b · Compare Raster display and vector display Lec 1 · p15–16

Raster	Vector
Composed of pixels	Composed of connected sequence of points (paths)
Produce large file size	Produce small file size
Provide more details shape	Provide smooth shape but limited to set of colors
Used for details images such as photo	Used for less details images such as logos

c · True / False

1. ✗ **False** Transformations can be composed, and their order does not matter.
   Correct: Transformations can be composed (multiplying matrices) but are NOT commutative, so proper order is vital. Lec 1 · p23
2. ✓ **True** A valuator input primitive is scalar values (typically between 0 and 1). Lec 1 · p13
3. ✗ **False** In display (physical) coordinates, Y increases up.
   Correct: Display coordinates: integer values only, X increasing right, Y increasing down (1 unit = 1 pixel). Lec 1 · p20
4. ✓ **True** OpenGL uses normalized device coordinates: the coordinates of each vertex should be between −1.0 and 1.0. Lec 1 · p21
5. ✗ **False** Enlarging a raster (bitmap) image keeps it sharp.
   Correct: Scaling down raster images is easy but enlarging a bitmap makes it pixelated or simply blurred. Lec 1 · p15

d · Complete

1. Window is _____ to be displayed, viewport is _____ to be displayed. → **what · where** Lec 3 · p3

2. In gray-scale images, N bit has _____ possible color values. → **2^N** Lec 2 · p23
3. From model to image: Model → _____ → View → _____ → Display. → **World · NDC** Lec 2 · p3

e · Explain 1) Explain the stages of the OpenGL graphics pipeline. Lec 1 · p24

- **Vertex shader:** define object vertices to move object around (translation, scaling, rotation).
- **Clipping:** cutting vertices that will not be on the screen.
- **Rasterization:** creating pixels for each part of the object.
- **Fragment shader:** add values for each pixel (coloring object shape).

2) How does a raster system display a straight line? Lec 4 · p7

- First project the endpoints to integer screen coordinates and determine the nearest pixel positions along the line path.
- Then the line color is loaded into the frame buffer at the corresponding pixel coordinates.
- Reading from the frame buffer, the video controller plots the screen pixels.

f · Calculate

1. If we use 10 bits pixel value in a 24-bit lookup table, how many entries does the LUT have and what is its size in bytes?
   → **Entries = 2^{10} = 1024 entry · size = (2^{10} × 24)/8 = 3072 byte** Lec 2 · p39
2. Window (W.l, W.r, W.b, W.t) = (20, 80, 40, 80), viewport (30, 60, 40, 60). Find (XV, YV) for (XW, YW) = (30, 80).
   → **Sx = 30/60 = 0.5 · Sy = 20/40 = 0.5 · XV = 30 + 0.5(30 − 20) = 35 · YV = 40 + 0.5(80 − 40) = 60 → (35, 60)** Lec 3 · p6

الفكرة الإجابة بجمل المحاضرة نفسها: التعريف جملة، والمقارنة جدول، والـ False لازم يتصحح.

Q2-1 Source: Lec 5 · p16–23 · Graphics_Sec 6 · p2

Question 2 (1) — Midpoint ellipse · $r_x = 6$ and $r_y = 4$

HOW TO SOLVE

1. Compute r_x^2, r_y^2 and $p1_0$.
2. Region 1: fill the table (increment x) until $2r_y^2x \geq 2r_x^2y$.
3. Compute $p2_0$ from the last point of region 1.
4. Region 2: fill the table (decrement y) until y = 0.
5. Plot the points.

$r_x^2 = 36$, $r_y^2 = 16$ · initial point (0, 4)
$p1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4}r_x^2 = 16 - 36\times4 + \frac{1}{4}\times36 = -119$

Region 1

Pk_region1	x	y	$2r_y^2x$	$2r_x^2y$
	0	4	0	288
−119	1	4	32	288

−71	2	4	64	288
9	3	3	96	216
−95	4	3	128	216
49	5	2	160	144

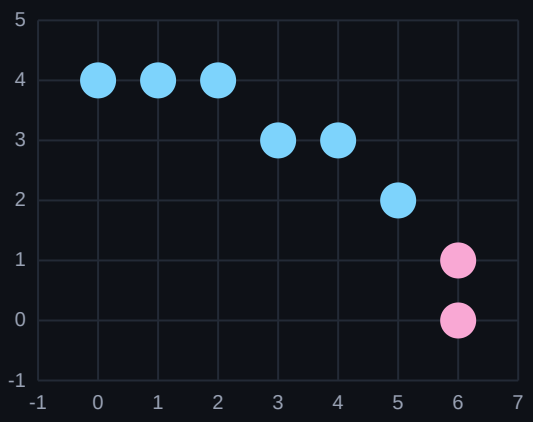
Region 1 stops when $2r_y^2x \geq 2r_x^2y$ → at (5, 2): 160 ≥ 144

Initial point of region 2 = last point of region 1 (5, 2) · $p2_0 = r_y^2(x_0 + \frac{1}{2})^2 + r_x^2(y_0 - 1)^2 - r_x^2r_y^2 = 16\times(5.5)^2 + 36\times(1)^2 - 36\times16 = -56$

Region 2

Pk_region2	x	y	$2r_y^2x$	$2r_x^2y$
	5	2	160	144
−56	6	1	192	72
100	6	0		

Region 1 (blue) · Region 2 (pink)



الفكرة Region 1 بنزوّد x لحد ما x_y^2r يبقى ≤ y_x^2r، وبعدين Region 2 بنقلّل y لحد الصفر.

Q2-2

Source: Lec 6 · p20–33 · Lec 1 · p22

Question 2 (2) — P′ = T(6, 3) · R(45) · S(3, 1) · T(−6, −3) · P

Read from right to left — the matrix nearest to P is applied first:

1. **T(−6, −3):** Translate the object so that the point (6, 3) moves to the origin.
2. **S(3, 1):** Scale around the origin: every x is multiplied by 3 (stretched 3 times horizontally), y does not change.
3. **R(45):** Rotate 45 degree around the origin (positive angle → counter-clockwise).
4. **T(6, 3):** Translate the object so that the point (6, 3) is back to its original position.

So the object is stretched 3 times horizontally and rotated 45° around the fixed point (6, 3).

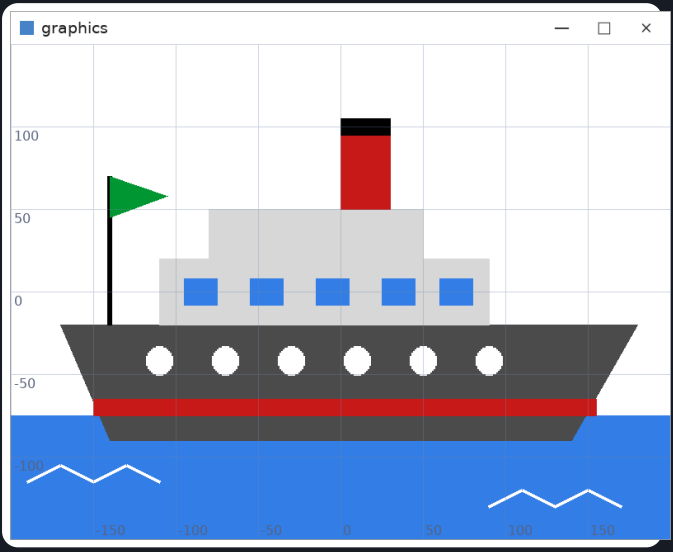
$$\begin{vmatrix} 1 & 0 & 6 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos 45 & -\sin 45 & 0 \\ \sin 45 & \cos 45 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 3 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & -6 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{vmatrix}$$

الفكرة اقرا الـ matrices من اليمين للشمال، وكل matrix = جملة واحدة بتوصف اللي حصل للشكل.

Q3

Source: main / myInit: Lec 3 · p10 · circleMidpoint: Lec 5 · p14

Question 3 — program



The output with the world-coordinate grid of gluOrtho2D(−200, 200, −150, 150) — read the vertices from it. Any coordinates that give the same figure are correct.

PROGRAM MAP — one file, written in this order

1	**Headers & prototypes**	libraries + declare the functions	lines 1–6
2	**main()**	create the window and start the program	lines 7–18
3	**myInit()**	background color + coordinate system	lines 19–26
4	**myDisplay()**	draws the figure, part by part	lines 27–155
5	**circleMidpoint()**	midpoint circle algorithm (Lec 5)	lines 156–174
6	**circleplot()**	plots the 8 symmetric points	lines 175–187

1 Headers & prototypes — libraries + declare the functions

START · line 1

```
1  #include <windows.h>
2  #include <GL/glut.h>
3  void myInit();
4  void myDisplay();
5  void circleMidpoint(int xc, int yc, int radius, GLenum mode);
6  void circleplot(int xc, int yc, int x, int y, GLenum mode);
```

END of Headers & prototypes · line 6

2 main() — create the window and start the program

START · line 7

```
7  int main(int argc, char** argv)
8  {
9      glutInit(&argc, argv);
10     glutInitDisplayMode(GLUT_RGB);//set display mode
```

```
11      glutInitWindowSize(400, 300);//set window size
12      glutInitWindowPosition(100, 150);//set window position
13      glutCreateWindow("graphics");
14      myInit();
15      glutDisplayFunc(myDisplay);
16      glutMainLoop();
17      return 0;
18  }
```

END of main() · line 18

3 myInit() background color + coordinate system

START · line 19

```
19  void myInit()
20  {
21      glClearColor(1.0, 1.0, 1.0, 0.0);
22      glColor3f(0.0f, 0.0f, 0.0f);
23      glMatrixMode(GL_PROJECTION);
24      glLoadIdentity();
25      gluOrtho2D(-200, 200, -150, 150);
26  }
```

END of myInit() · line 26

4 myDisplay() draws the figure, part by part

START · line 27

```
27  void myDisplay()
28  {
29      glClear(GL_COLOR_BUFFER_BIT);
30
31      // sea
32      glColor3f(0.2, 0.5, 0.9);
33      glBegin(GL_QUADS);
34      glVertex2f(-200, -150);
35      glVertex2f(200, -150);
36      glVertex2f(200, -75);
37      glVertex2f(-200, -75);
38      glEnd();
39
40      // waves
41      glColor3f(1.0, 1.0, 1.0);
42      glLineWidth(2);
43      glBegin(GL_LINE_STRIP);
44      glVertex2f(-190, -115);
45      glVertex2f(-170, -105);
46      glVertex2f(-150, -115);
47      glVertex2f(-130, -105);
48      glVertex2f(-110, -115);
49      glEnd();
50      glBegin(GL_LINE_STRIP);
51      glVertex2f(90, -130);
52      glVertex2f(110, -120);
53      glVertex2f(130, -130);
54      glVertex2f(150, -120);
55      glVertex2f(170, -130);
56      glEnd();
57
```

```
    // hull
    glColor3f(0.3, 0.3, 0.3);
    glBegin(GL_POLYGON);
    glVertex2f(-170, -20);
    glVertex2f(180, -20);
    glVertex2f(140, -90);
    glVertex2f(-140, -90);
    glEnd();

    // red line on the hull
    glColor3f(0.8, 0.1, 0.1);
    glBegin(GL_QUADS);
    glVertex2f(-150, -75);
    glVertex2f(155, -75);
    glVertex2f(155, -65);
    glVertex2f(-150, -65);
    glEnd();

    // portholes
    glColor3f(1.0, 1.0, 1.0);
    circleMidpoint(-110, -42, 8, GL_LINES);
    circleMidpoint(-70, -42, 8, GL_LINES);
    circleMidpoint(-30, -42, 8, GL_LINES);
    circleMidpoint(10, -42, 8, GL_LINES);
    circleMidpoint(50, -42, 8, GL_LINES);
    circleMidpoint(90, -42, 8, GL_LINES);

    // cabins
    glColor3f(0.8, 0.8, 0.8);
    glBegin(GL_QUADS);
    glVertex2f(-110, -20);
    glVertex2f(90, -20);
    glVertex2f(90, 20);
    glVertex2f(-110, 20);
    glEnd();
    glBegin(GL_QUADS);
    glVertex2f(-80, 20);
    glVertex2f(50, 20);
    glVertex2f(50, 50);
    glVertex2f(-80, 50);
    glEnd();

    // cabin windows
    glColor3f(0.2, 0.5, 0.9);
    glBegin(GL_QUADS);
    glVertex2f(-95, -8);
    glVertex2f(-75, -8);
    glVertex2f(-75, 8);
    glVertex2f(-95, 8);
    glVertex2f(-55, -8);
    glVertex2f(-35, -8);
    glVertex2f(-35, 8);
    glVertex2f(-55, 8);
    glVertex2f(-15, -8);
    glVertex2f(5, -8);
    glVertex2f(5, 8);
    glVertex2f(-15, 8);
    glVertex2f(25, -8);
    glVertex2f(45, -8);
```

```
    glVertex2f(45, 8);
    glVertex2f(25, 8);
    glVertex2f(60, -8);
    glVertex2f(80, -8);
    glVertex2f(80, 8);
    glVertex2f(60, 8);
    glEnd();

    // chimney
    glColor3f(0.8, 0.1, 0.1);
    glBegin(GL_QUADS);
    glVertex2f(0, 50);
    glVertex2f(30, 50);
    glVertex2f(30, 95);
    glVertex2f(0, 95);
    glEnd();
    glColor3f(0.0, 0.0, 0.0);
    glBegin(GL_QUADS);
    glVertex2f(0, 95);
    glVertex2f(30, 95);
    glVertex2f(30, 105);
    glVertex2f(0, 105);
    glEnd();

    // mast and flag
    glLineWidth(3);
    glBegin(GL_LINES);
    glVertex2f(-140, -20);
    glVertex2f(-140, 70);
    glEnd();
    glColor3f(0.0, 0.6, 0.2);
    glBegin(GL_TRIANGLES);
    glVertex2f(-140, 70);
    glVertex2f(-140, 45);
    glVertex2f(-105, 58);
    glEnd();

    glFlush();
}
```

END of myDisplay() · line 155

5 `circleMidpoint()` midpoint circle algorithm (Lec 5)

START · line 156

```
void circleMidpoint(int xc, int yc, int radius, GLenum mode)
{
    int x = 0;
    int y = radius;
    int p0 = 1 - radius;
    circleplot(xc, yc, x, y, mode);
    while (x < y)
    {
        x++;
        if (p0 < 0)
            p0 = p0 + 2 * x + 1;
        else
        {
            y--;
```

```
170             p0 = p0 + 2 * x -2*y + 1;
171         }
172         circleplot(xc, yc, x, y, mode);
173     }
174 }
```

END of circleMidpoint() · line 174

6 circleplot() plots the 8 symmetric points

START · line 175

```
175 void circleplot(int xc, int yc, int x, int y, GLenum mode)
176 {
177     glBegin(mode);
178     glVertex2f(xc + x, yc + y);
179     glVertex2f(xc - x, yc + y);
180     glVertex2f(xc + x, yc - y);
181     glVertex2f(xc - x, yc - y);
182     glVertex2f(xc + y, yc + x);
183     glVertex2f(xc - y, yc + x);
184     glVertex2f(xc + y, yc - x);
185     glVertex2f(xc - y, yc - x);
186     glEnd();
187 }
```

END of circleplot() · line 187

الفكرة البرنامج ملف واحد مقسوم functions: كل function بتبدأ عند START وتخلص عند END. وجوه myDisplay كل جزء من الصورة بيبدأ بـ comment ملوّن.

CG / MODEL ANSWER / MODEL 02

Model Answer — Model 2

M2

Exam paper on page 16 · each answer shows the lecture page of its method

Q1

Source: the page is next to each answer

Question 1 — theory

a · Define

Aliasing: The jagged, or stair-step, appearance of raster primitives: distortion of information due to low-frequency sampling (under sampling). Lec 5 · p24

Pixel and scan line: Pixel: one dot of the raster · Scan line: a row of pixels. Lec 2 · p25

Unit circle: The circle of radius 1 centered at the origin (0, 0) in the Cartesian coordinate system. Lec 5 · p4

b · Compare Interactive and non-interactive computer graphics Lec 1 · p11

Non-interactive	Interactive
The user does not have any control over the image	The user can make any change by editing with an input device
One-way communication between computer and user	Two-way communication between computer and user
Example: animation movies	Example: games

c · True / False

1. ✗ **False** Image processing is concerned with the creation of images.
   Correct: Computer graphics is concerned with the creation of images; image processing analyzes pictures to derive descriptions.
   Lec 1 · p6
2. ✓ **True** Bresenham's line algorithm uses only incremental integer calculations. Lec 4 · p25
3. ✗ **False** A true color image has a color depth of 16 bit.
   Correct: True color image has a color depth of 24 bit (8 bit for each color).
   Lec 2 · p24
4. ✓ **True** A special case arises when a line segment shrinks to a single point, and it is drawn as a "dot".
   Lec 2 · p13
5. ✗ **False** In DDA, if m > 1 we sample at unit x intervals.
   Correct: If m > 1 (dy > dx) we sample at unit y intervals: $x_{k+1} = x_k + 1/m$.
   Lec 4 · p17

d · Complete

1. Geometric model describes _____, while mathematical model describes _____. → **how objects appears (color, texture or shape) · how object moves** Lec 1 · p9
2. Polygon is a closed polyline where its _____ equal to its _____. → **start point · last point** Lec 2 · p12

3. Text attributes are font face, _____, size, style, spacing and _____. → **weight · orientation** Lec 2 · p15

e · Explain 1) List the advantages and disadvantages of the DDA algorithm. Lec 4 · p24

- **Advantages:** it is a simple algorithm to implement · it is a faster algorithm than the direct line equation.
- **Disadvantages:** the floating-point arithmetic is time-consuming · the method of round-off is also time-consuming · the accumulation of round-off error can cause the pixel positions to drift away from the true line path for long line segments.

2) What are the trivial situations that do not need line-drawing algorithms? Lec 4 · p10

- m = 0 → horizontal line.
- m = ∞ → vertical line.
- m = 1 → the line y = x.

f · Calculate

1. If we use direct coding of RGB value with 3 bits per primary color, how many possible colors do we have for each pixel?
   → **$2^3 \times 2^3 \times 2^3 = 512$ color** Lec 2 · p39
2. For Bresenham's line from (20, 10) to (30, 18), find p0 and the two update values.
   → **Δx = 10, Δy = 8 · p0 = 2Δy − Δx = 6 · p ≥ 0: p + 2Δy − 2Δx = p − 4 · p < 0: p + 2Δy = p + 16** Lec 4 · p30

الفكرة الإجابة بجمل المحاضرة نفسها: التعريف جملة، والمقارنة جدول، والـ False لازم يتصحح.

Q2-1

Source: Lec 5 · p11–13 · Graphics_Sec 6 · p1

Question 2 (1) — Midpoint circle · r = 14

HOW TO SOLVE

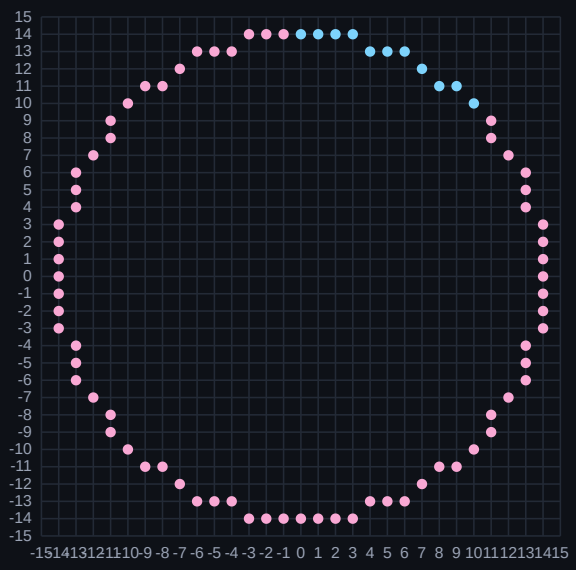
1. P0 = 1 − r, start at (0, r).
2. Use the sign of P to choose y or y − 1, then update P.
3. Stop when x ≥ y.
4. Plot the octant and its symmetry.

Initial point (0, 14) · P0 = 1 − r = -13

P	X	Y	$2x_{k+1}$	$2y_{k+1}$
-	0	14	-	-
−13	1	14	2	28
−10	2	14	4	28
−5	3	14	6	28
2	4	13	8	26
−15	5	13	10	26
−4	6	13	12	26

Octant points (blue) + eight-way symmetry (pink)

9	7	12	14	24
0	8	11	16	22
−5	9	11	18	22
14	10	10	-	-



الفكرة إشارة P بتحدد y هيفضل ثابت ولا هينزل 1، وبنقف لما x يوصل لـ y.

Q2-2

Source: Lec 6 · p20–33 · Lec 1 · p22

Question 2 (2) — P′ = R(30) · T(5, −2) · S(0.5, 0.5) · P

Read from right to left — the matrix nearest to P is applied first:

1. **S(0.5, 0.5):** Scale around the origin: the object shrinks to half its size.
2. **T(5, −2):** Translate the object 5 units to the right and 2 units down.
3. **R(30):** Rotate 30 degree counter-clockwise around the origin (the object turns around the origin, not around itself).

So the object is made half its size, moved by (5, −2), then rotated 30° around the origin.

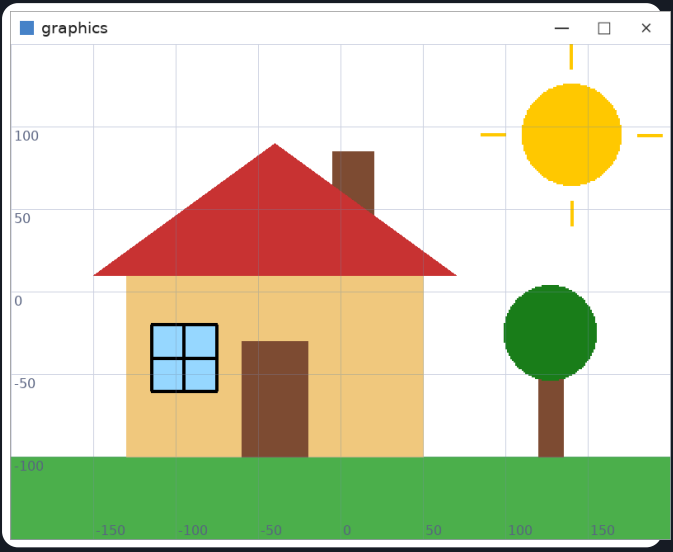
$$\begin{vmatrix} \cos 30 & -\sin 30 & 0 \\ \sin 30 & \cos 30 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & 5 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 0.5 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

الفكرة اقرا الـ matrices من اليمين للشمال، وكل matrix = جملة واحدة بتوصف اللي حصل للشكل.

Q3

Source: main / myInit: Lec 3 · p10 · circleMidpoint: Lec 5 · p14

Question 3 — program



The output with the world-coordinate grid of gluOrtho2D(−200, 200, −150, 150) — read the vertices from it. Any coordinates that give the same figure are correct.

PROGRAM MAP — one file, written in this order

1	**Headers & prototypes**	libraries + declare the functions	lines 1–6
2	**main()**	create the window and start the program	lines 7–18
3	**myInit()**	background color + coordinate system	lines 19–26
4	**myDisplay()**	draws the figure, part by part	lines 27–125
5	**circleMidpoint()**	midpoint circle algorithm (Lec 5)	lines 126–144
6	**circleplot()**	plots the 8 symmetric points	lines 145–157

1 Headers & prototypes — libraries + declare the functions

START · line 1

```
1  #include <windows.h>
2  #include <GL/glut.h>
3  void myInit();
4  void myDisplay();
5  void circleMidpoint(int xc, int yc, int radius, GLenum mode);
6  void circleplot(int xc, int yc, int x, int y, GLenum mode);
```

END of Headers & prototypes · line 6

2 main() — create the window and start the program

START · line 7

```
7  int main(int argc, char** argv)
8  {
9      glutInit(&argc, argv);
10     glutInitDisplayMode(GLUT_RGB);//set display mode
```

```
11      glutInitWindowSize(400, 300);//set window size
12      glutInitWindowPosition(100, 150);//set window position
13      glutCreateWindow("graphics");
14      myInit();
15      glutDisplayFunc(myDisplay);
16      glutMainLoop();
17      return 0;
18  }
```

END of main() · line 18

3 myInit() background color + coordinate system

START · line 19

```
19  void myInit()
20  {
21      glClearColor(1.0, 1.0, 1.0, 0.0);
22      glColor3f(0.0f, 0.0f, 0.0f);
23      glMatrixMode(GL_PROJECTION);
24      glLoadIdentity();
25      gluOrtho2D(-200, 200, -150, 150);
26  }
```

END of myInit() · line 26

4 myDisplay() draws the figure, part by part

START · line 27

```
27  void myDisplay()
28  {
29      glClear(GL_COLOR_BUFFER_BIT);
30
31      // grass
32      glColor3f(0.3, 0.7, 0.3);
33      glBegin(GL_QUADS);
34      glVertex2f(-200, -150);
35      glVertex2f(200, -150);
36      glVertex2f(200, -100);
37      glVertex2f(-200, -100);
38      glEnd();
39
40      // wall
41      glColor3f(0.9, 0.8, 0.5);
42      glBegin(GL_QUADS);
43      glVertex2f(-130, -100);
44      glVertex2f(50, -100);
45      glVertex2f(50, 10);
46      glVertex2f(-130, 10);
47      glEnd();
48
49      // chimney
50      glColor3f(0.5, 0.3, 0.2);
51      glBegin(GL_QUADS);
52      glVertex2f(-5, 40);
53      glVertex2f(20, 40);
54      glVertex2f(20, 85);
55      glVertex2f(-5, 85);
56      glEnd();
57
```

```
    // roof
    glColor3f(0.8, 0.2, 0.2);
    glBegin(GL_TRIANGLES);
    glVertex2f(-150, 10);
    glVertex2f(70, 10);
    glVertex2f(-40, 90);
    glEnd();

    // door
    glColor3f(0.5, 0.3, 0.2);
    glBegin(GL_QUADS);
    glVertex2f(-60, -100);
    glVertex2f(-20, -100);
    glVertex2f(-20, -30);
    glVertex2f(-60, -30);
    glEnd();

    // window
    glColor3f(0.6, 0.8, 1.0);
    glBegin(GL_QUADS);
    glVertex2f(-115, -60);
    glVertex2f(-75, -60);
    glVertex2f(-75, -20);
    glVertex2f(-115, -20);
    glEnd();
    glColor3f(0.0, 0.0, 0.0);
    glLineWidth(2);
    glBegin(GL_LINE_LOOP);
    glVertex2f(-115, -60);
    glVertex2f(-75, -60);
    glVertex2f(-75, -20);
    glVertex2f(-115, -20);
    glEnd();
    glBegin(GL_LINES);
    glVertex2f(-95, -60);
    glVertex2f(-95, -20);
    glVertex2f(-115, -40);
    glVertex2f(-75, -40);
    glEnd();

    // sun
    glColor3f(1.0, 0.8, 0.0);
    circleMidpoint(140, 95, 30, GL_LINES);
    glLineWidth(2);
    glBegin(GL_LINES);
    glVertex2f(140, 135);
    glVertex2f(140, 150);
    glVertex2f(140, 55);
    glVertex2f(140, 40);
    glVertex2f(100, 95);
    glVertex2f(85, 95);
    glVertex2f(180, 95);
    glVertex2f(195, 95);
    glEnd();

    // tree
    glColor3f(0.5, 0.3, 0.2);
    glBegin(GL_QUADS);
    glVertex2f(120, -100);
```

```
    glVertex2f(135, -100);
    glVertex2f(135, -40);
    glVertex2f(120, -40);
    glEnd();
    glColor3f(0.1, 0.5, 0.1);
    circleMidpoint(127, -25, 28, GL_LINES);

    glFlush();
}
```

END of myDisplay() · line 125

5 circleMidpoint() midpoint circle algorithm (Lec 5)

START · line 126

```
void circleMidpoint(int xc, int yc, int radius, GLenum mode)
{
    int x = 0;
    int y = radius;
    int p0 = 1 - radius;
    circleplot(xc, yc, x, y, mode);
    while (x < y)
    {
        x++;
        if (p0 < 0)
            p0 = p0 + 2 * x + 1;
        else
        {
            y--;
            p0 = p0 + 2 * x -2*y + 1;
        }
        circleplot(xc, yc, x, y, mode);
    }
}
```

END of circleMidpoint() · line 144

6 circleplot() plots the 8 symmetric points

START · line 145

```
void circleplot(int xc, int yc, int x, int y, GLenum mode)
{
    glBegin(mode);
    glVertex2f(xc + x, yc + y);
    glVertex2f(xc - x, yc + y);
    glVertex2f(xc + x, yc - y);
    glVertex2f(xc - x, yc - y);
    glVertex2f(xc + y, yc + x);
    glVertex2f(xc - y, yc + x);
    glVertex2f(xc + y, yc - x);
    glVertex2f(xc - y, yc - x);
    glEnd();
}
```

END of circleplot() · line 157

الفكرة البرنامج ملف واحد مقسوم functions: كل function بتبدأ عند START وتخلص عند END. وجوه myDisplay كل جزء من الصورة بيبدأ بـ comment ملوّن.

CG / MODEL ANSWER / MODEL 03

Model Answer — Model 3

M3

Exam paper on page 18 · each answer shows the lecture page of its method

Q1

Source: the page is next to each answer

Question 1 — theory

a · Define

Polyline: Connected sequence of straight lines; each line is called an edge and two adjacent lines meet at a vertex. Lec 2 · p12

Window to viewport mapping: The process of mapping or transforming a 2D world coordinate scene to device coordinates. Lec 3 · p3

Vector: Vectors have magnitude and direction; they represent displacements between points, or directions. Lec 4 · p3

b · Compare Direct coding and Look-Up Table (LUT) Lec 2 · p35–36 · Graphics_SEC 3 · p1

Direct coding	Indexed color with LUT
Represents each pixel's color directly	Pixel holds only its index into the palette
Stores the full color specification in each pixel (e.g. 24 bits for true color)	Color information is stored in a separate color lookup table (CLUT) or palette
More memory	Compromise between lower storage requirement and enough number of colors

c · True / False

1. ✗ **False** When the GPU frame rate is higher than the monitor refresh rate, stuttering happens.
   Correct: Frame rate > refresh rate can cause screen tearing; frame rate < refresh rate leads to stuttering.
   Lec 2 · p28
2. ✓ **True** Linear transformations preserve co-linearity and ratios of distances. Lec 6 · p3
3. ✗ **False** Scaling preserves the angles between lines for any s_x, s_y.
   Correct: Scaling doesn't preserve angles between lines except with uniform scaling $s_x = s_y$.
   Lec 6 · p6
4. ✗ **False** Hand designed images are made by digitizing a photograph.
   Correct: Digitizing a photograph or television image gives scanned images; hand designed images: a designer figures out the value of each cell (paint program).
   Lec 2 · p20
5. ✓ **True** The attributes of a filled region are enclosing border, pattern and color. Lec 2 · p17

d · Complete

1. Output primitives (elements of pictures) are _____, _____, _____ and _____. → **polyline · text · filled regions · raster images** Lec 2 · p10
2. Each integer screen position references the _____ of a pixel area. → **center** Lec 4 · p5
3. setPixel stores the current color setting into the frame buffer at integer position (x, y); _____ retrieves the current frame-buffer setting for a pixel location. → **getPixel** Lec 4 · p6

e · Explain 1) How are raster images created? Lec 2 · p20

- **Hand designed images:** a designer figures out what values are needed for each cell and stores it in memory (paint program).
- **Computed images:** an algorithm is used to "render" a scene modeled abstractly in computer memory.
- **Scanned images:** a photograph or television image can be digitized.

2) List the geometric transformation functions. Lec 6 · p2

- Translation
- Scaling
- Rotation
- Reflection
- Shearing

f · Calculate

1. If we use direct coding of RGB value with 5 bits per primary color, how many possible colors do we have?
   → **$2^5 \times 2^5 \times 2^5 = 32 \times 32 \times 32 = 32{,}768$ colors** Graphics_SEC 3 · p2
2. Find the matching viewport for a world window with aspect ratio R = 1.6 when the screen window has width 360 and height 200.
   → **W/H = 1.8 > R → height = H = 200, width = H·R = 320 → glViewport(0, 0, 320, 200)** Lec 3 · p26

الفكرة الإجابة بجمل المحاضرة نفسها: التعريف جملة، والمقارنة جدول، والـ False لازم يتصحح.

Q2-1 Source: Lec 4 · p28–30 · Graphics sec5 · p2

Question 2 (1) — Bresenham line · (15, 20) and (25, 26)

HOW TO SOLVE

1. Compute Δx, Δy and p0 = 2Δy − Δx.
2. Find the two update values: + 2Δy and + 2Δy − 2Δx.
3. Fill the table: the sign of P decides if y increases.
4. Plot the pixels.

$|\Delta x| = |25 - 15| = 10$, $|\Delta y| = |26 - 20| = 6$ · $|\Delta x| > |\Delta y|$ · $p0 = 2\Delta y - \Delta x = 12 - 10 = 2$ · $P_k < 0$: x incremented, y remains, $P_{k+1} = P_k + 12$ · $P_k \geq 0$: x and y incremented, $P_{k+1} = P_k - 8$

P	X	Y	SetPixel(x, y)
-	15	20	(15,20)

Plotted pixels

2	16	21	(16,21)
−6	17	21	(17,21)
6	18	22	(18,22)
−2	19	22	(19,22)
10	20	23	(20,23)
2	21	24	(21,24)
−6	22	24	(22,24)
6	23	25	(23,25)
−2	24	25	(24,25)
10	25	26	(25,26)



الفكرة الـ P السالب يخلّي y ثابت، والموجب (أو الصفر) يزوّد y بواحد.

Q2-2

Source: Lec 6 · p20–33 · Lec 1 · p22

Question 2 (2) — P′ = T(−2, 4) · S(1, −1) · R(−60) · P

Read from right to left — the matrix nearest to P is applied first:

1. **R(−60):** Rotate 60 degree around the origin; the angle is negative → clockwise.
2. **S(1, −1):** Scale with s_x = 1 and s_y = −1: x stays, y → −y — a mirror image (reflection) across the x-axis.
3. **T(−2, 4):** Translate the object 2 units to the left and 4 units up.

So the object is rotated 60° clockwise about the origin, flipped upside-down across the x-axis, then moved by (−2, 4).

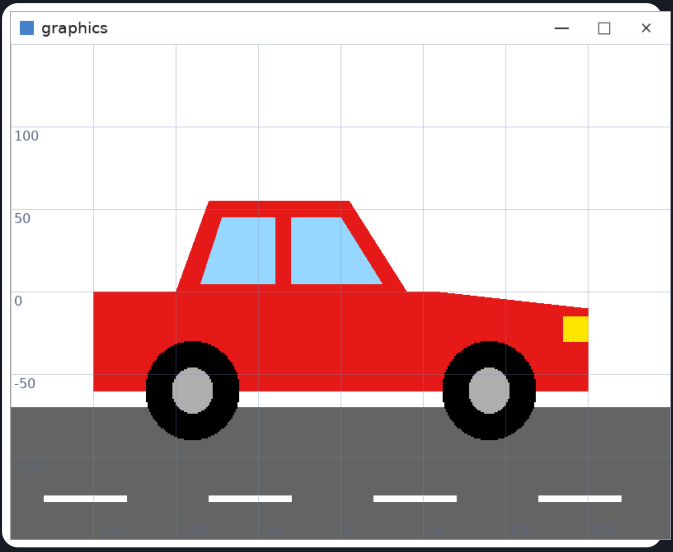
$$\begin{vmatrix} 1 & 0 & -2 \\ 0 & 1 & 4 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos -60 & -\sin -60 & 0 \\ \sin -60 & \cos -60 & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

الفكرة اقرا الـ matrices من اليمين للشمال، وكل matrix = جملة واحدة بتوصف اللي حصل للشكل.

Q3

Source: main / myInit: Lec 3 · p10 · circleMidpoint: Lec 5 · p14

Question 3 — program



The output with the world-coordinate grid of gluOrtho2D(−200, 200, −150, 150) — read the vertices from it. Any coordinates that give the same figure are correct.

PROGRAM MAP — one file, written in this order

1	**Headers & prototypes**	libraries + declare the functions	lines 1–6
2	**main()**	create the window and start the program	lines 7–18
3	**myInit()**	background color + coordinate system	lines 19–26
4	**myDisplay()**	draws the figure, part by part	lines 27–103
5	**circleMidpoint()**	midpoint circle algorithm (Lec 5)	lines 104–122
6	**circleplot()**	plots the 8 symmetric points	lines 123–135

1 Headers & prototypes — libraries + declare the functions

START · line 1

```
1  #include <windows.h>
2  #include <GL/glut.h>
3  void myInit();
4  void myDisplay();
5  void circleMidpoint(int xc, int yc, int radius, GLenum mode);
6  void circleplot(int xc, int yc, int x, int y, GLenum mode);
```

END of Headers & prototypes · line 6

2 main() — create the window and start the program

START · line 7

```
7  int main(int argc, char** argv)
8  {
9      glutInit(&argc, argv);
10     glutInitDisplayMode(GLUT_RGB);//set display mode
```

```
11      glutInitWindowSize(400, 300);//set window size
12      glutInitWindowPosition(100, 150);//set window position
13      glutCreateWindow("graphics");
14      myInit();
15      glutDisplayFunc(myDisplay);
16      glutMainLoop();
17      return 0;
18  }
```

END of main() · line 18

3 myInit() background color + coordinate system

START · line 19

```
19  void myInit()
20  {
21      glClearColor(1.0, 1.0, 1.0, 0.0);
22      glColor3f(0.0f, 0.0f, 0.0f);
23      glMatrixMode(GL_PROJECTION);
24      glLoadIdentity();
25      gluOrtho2D(-200, 200, -150, 150);
26  }
```

END of myInit() · line 26

4 myDisplay() draws the figure, part by part

START · line 27

```
27  void myDisplay()
28  {
29      glClear(GL_COLOR_BUFFER_BIT);
30
31      // road
32      glColor3f(0.4, 0.4, 0.4);
33      glBegin(GL_QUADS);
34      glVertex2f(-200, -150);
35      glVertex2f(200, -150);
36      glVertex2f(200, -70);
37      glVertex2f(-200, -70);
38      glEnd();
39      glColor3f(1.0, 1.0, 1.0);
40      glLineWidth(4);
41      glBegin(GL_LINES);
42      glVertex2f(-180, -125);
43      glVertex2f(-130, -125);
44      glVertex2f(-80, -125);
45      glVertex2f(-30, -125);
46      glVertex2f(20, -125);
47      glVertex2f(70, -125);
48      glVertex2f(120, -125);
49      glVertex2f(170, -125);
50      glEnd();
51
52      // car body
53      glColor3f(0.9, 0.1, 0.1);
54      glBegin(GL_POLYGON);
55      glVertex2f(-150, -60);
56      glVertex2f(150, -60);
57      glVertex2f(150, -10);
```

```
    glVertex2f(60, 0);
    glVertex2f(-150, 0);
    glEnd();

    // cabin
    glBegin(GL_POLYGON);
    glVertex2f(-100, 0);
    glVertex2f(40, 0);
    glVertex2f(5, 55);
    glVertex2f(-80, 55);
    glEnd();

    // windows
    glColor3f(0.6, 0.8, 1.0);
    glBegin(GL_POLYGON);
    glVertex2f(-85, 5);
    glVertex2f(-40, 5);
    glVertex2f(-40, 45);
    glVertex2f(-72, 45);
    glEnd();
    glBegin(GL_POLYGON);
    glVertex2f(-30, 5);
    glVertex2f(25, 5);
    glVertex2f(0, 45);
    glVertex2f(-30, 45);
    glEnd();

    // head light
    glColor3f(1.0, 0.9, 0.0);
    glBegin(GL_QUADS);
    glVertex2f(135, -30);
    glVertex2f(150, -30);
    glVertex2f(150, -15);
    glVertex2f(135, -15);
    glEnd();

    // wheels
    glColor3f(0.0, 0.0, 0.0);
    circleMidpoint(-90, -60, 28, GL_LINES);
    circleMidpoint(90, -60, 28, GL_LINES);
    glColor3f(0.7, 0.7, 0.7);
    circleMidpoint(-90, -60, 12, GL_LINES);
    circleMidpoint(90, -60, 12, GL_LINES);

    glFlush();
}
```

END of myDisplay() · line 103

5 **circleMidpoint()** midpoint circle algorithm (Lec 5) START · line 104

```
void circleMidpoint(int xc, int yc, int radius, GLenum mode)
{
    int x = 0;
    int y = radius;
    int p0 = 1 - radius;
    circleplot(xc, yc, x, y, mode);
    while (x < y)
```

```
111     {
112         x++;
113         if (p0 < 0)
114             p0 = p0 + 2 * x + 1;
115         else
116         {
117             y--;
118             p0 = p0 + 2 * x -2*y + 1;
119         }
120         circleplot(xc, yc, x, y, mode);
121     }
122 }
```

END of circleMidpoint() · line 122

6 circleplot() plots the 8 symmetric points

START · line 123

```
123 void circleplot(int xc, int yc, int x, int y, GLenum mode)
124 {
125     glBegin(mode);
126     glVertex2f(xc + x, yc + y);
127     glVertex2f(xc - x, yc + y);
128     glVertex2f(xc + x, yc - y);
129     glVertex2f(xc - x, yc - y);
130     glVertex2f(xc + y, yc + x);
131     glVertex2f(xc - y, yc + x);
132     glVertex2f(xc + y, yc - x);
133     glVertex2f(xc - y, yc - x);
134     glEnd();
135 }
```

END of circleplot() · line 135

الفكرة البرنامج ملف واحد مقسوم functions: كل function بتبدأ عند START وتخلص عند END. وجوه myDisplay كل جزء من الصورة بيبدأ بـ comment ملوّن.

CG / MODEL ANSWER / MODEL 04

Model Answer — Model 4

M4

Exam paper on page 20 · each answer shows the lecture page of its method

Q1

Source: the page is next to each answer

Question 1 — theory

a · Define

Graphics library (API): A set of procedures for handling graphics such as OpenGL, Direct3D or Java3D; it is intermediary between application and display hardware. Lec 1 · p17

Bitmap: A raster image is stored as a 2D array of numerical values; the array as a whole is often called a "pixel map" or "bitmap". Lec 2 · p19

Conceptual framework of computer graphics: How a graphics system is organized, what pieces exist and how they fit together. SEC 1 and 2 · p2

b · Compare Emissive and non-emissive displays Lec 2 · p27

Emissive	Non-emissive
Devices that convert electrical energy into light	Use optical effects to convert light from some other source into graphics patterns
Example: Light Emitting Diode (LED)	Example: Liquid Crystal Display (LCD)

c · True / False

1. ✗ **False** A concave polygon has all its diagonals inside the polygon.
   Correct: Concave polygon has at least one diagonal lying outside the polygon; convex has all diagonals inside.
   Graphics_SEC 3 · p1
2. ✓ **True** Positive angles rotate counter-clockwise about the origin. Lec 1 · p22
3. ✗ **False** In the midpoint circle algorithm, if P > 0 the midpoint is inside the circle.
   Correct: P > 0 → the midpoint is outside the circle; P < 0 → inside; P = 0 → on the circle.
   Lec 5 · p9, p12
4. ✓ **True** Nowadays the entire frame of pixel data is sent from the graphics card to the monitor digitally (HDMI, DisplayPort), row by row. Lec 2 · p29
5. ✗ **False** In a polyline, each line is called a vertex.
   Correct: Each line is called an edge; two adjacent lines meet at a vertex.
   Lec 2 · p12

d · Complete

1. Locator input primitive is _____; pick input primitive is _____. → **coordinate position (x, y) · selection of a part of the screen** Lec 1 · p13–14
2. In region 1 of the ellipse we increment _____ and calculate _____. → **x · y** Lec 5 · p17

3. The most familiar raster displays are flat panels, which include _____, _____, digital signage and automotive infotainment screens. → **smartphones and tablets · smart TVs** Lec 2 · p26

e · Explain 1) Explain the steps of transforming coordinates from modeling to device coordinates. Lec 2 · p9

- Object shapes are defined within their own modeling-coordinate systems.
- These shapes are placed within a global world-coordinate scene.
- The world-coordinate data is processed through the viewing pipeline into viewing and projection coordinates, then normalized coordinates.
- Device drivers take the normalized-coordinate representation and send it to the output devices for display.

2) Explain the three types of 2D reflection. Lec 6 · p7

- **Horizontal:** negative factor $s_y = -1$ → $x' = x$, $y' = -y$.
- **Vertical:** negative factor $s_x = -1$ → $x' = -x$, $y' = y$.
- **Diagonal:** $s_x = -1$ and $s_y = -1$ → $x' = -x$, $y' = -y$.

f · Calculate

1. A frame buffer supports 12 planes of memory (4 planes for each color). Calculate the total number of colors.
   → **$2^4 = 16$ levels per color → $16 \times 16 \times 16 = 4{,}096$ colors** Graphics_SEC 3 · p2
2. Using DDA, find the first three pixels of the line from (9, 8) to (2, 3).
   → **m ≤ 1, steps = −7 → x − 1, y − m: (8, 7.2) → (8, 7) · (7, 6.5) → (7, 7) · (6, 5.8) → (6, 6)** Lec 4 · p23

الفكرة الإجابة بجمل المحاضرة نفسها: التعريف جملة، والمقارنة جدول، والـ False لازم يتصحح.

Q2-1 Source: Lec 5 · p16–23 · Graphics_Sec 6 · p2

Question 2 (1) — Midpoint ellipse · $r_x = 10$ and $r_y = 6$

HOW TO SOLVE

1. Compute r_x^2, r_y^2 and $p1_0$.
2. Region 1: fill the table (increment x) until $2r_y^2x \geq 2r_x^2y$.
3. Compute $p2_0$ from the last point of region 1.
4. Region 2: fill the table (decrement y) until y = 0.
5. Plot the points.

$r_x^2 = 100$, $r_y^2 = 36$ · initial point (0, 6)

$p1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4}r_x^2 = 36 - 100\times6 + \frac{1}{4}\times100 = -539$

Region 1

Pk_region1	x	y	$2r_y^2x$	$2r_x^2y$
	0	6	0	1200
−539	1	6	72	1200
−431	2	6	144	1200

−251	3	6	216	1200
1	4	5	288	1000
−675	5	5	360	1000
−279	6	5	432	1000
189	7	4	504	800
−71	8	4	576	800
541	9	3	648	600

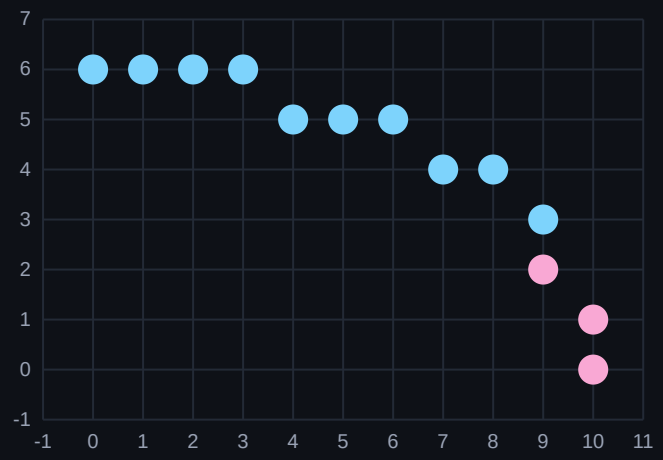
Region 1 stops when $2r_y^2x \geq 2r_x^2y$ → at (9, 3): 648 ≥ 600

Initial point of region 2 = last point of region 1 (9, 3) · $p2_0 = r_y^2(x_0 + \frac{1}{2})^2 + r_x^2(y_0 - 1)^2 - r_x^2r_y^2 = 36\times(9.5)^2 + 100\times(2)^2 - 100\times36 = 49$

Region 2

Pk_region2	x	y	$2r_y^2x$	$2r_x^2y$
	9	3	648	600
49	9	2	648	400
−251	10	1	720	200
369	10	0		

Region 1 (blue) · Region 2 (pink)



الفكرة Region 1 بنزوّد x لحد ما $x_y^2 2r$ يبقى ≤ $y_x^2 2r$، وبعدين Region 2 بنقلّل y لحد الصفر.

Q2-2

Source: Lec 6 · p20–33 · Lec 1 · p22

Question 2 (2) — P′ = T(3, 3) · R(90) · T(−3, −3) · T(4, 0) · P

Read from right to left — the matrix nearest to P is applied first:

1. **T(4, 0):** Translate the object 4 units to the right.
2. **T(−3, −3):** Translate so that the point (3, 3) moves to the origin.
3. **R(90):** Rotate 90 degree counter-clockwise around the origin.
4. **T(3, 3):** Translate back so that the point (3, 3) returns to its original position.

So the object is moved 4 units right, then rotated 90° around the point (3, 3). (The two first translations add: T(−3, −3)·T(4, 0) = T(1, −3).)

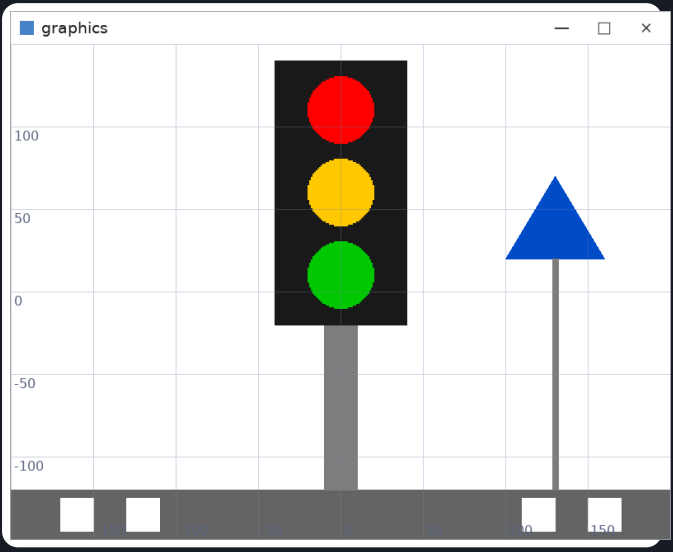
$$\begin{vmatrix} 1 & 0 & 3 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos 90 & -\sin 90 & 0 \\ \sin 90 & \cos 90 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & -3 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & 4 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

الفكرة اقرا الـ matrices من اليمين للشمال، وكل matrix = جملة واحدة بتوصف اللي حصل للشكل.

Q3

Source: main / myInit: Lec 3 · p10 · circleMidpoint: Lec 5 · p14

Question 3 — program



The output with the world-coordinate grid of gluOrtho2D(−200, 200, −150, 150) — read the vertices from it. Any coordinates that give the same figure are correct.

PROGRAM MAP — one file, written in this order

1	Headers & prototypes	libraries + declare the functions	lines 1–6
2	main()	create the window and start the program	lines 7–18
3	myInit()	background color + coordinate system	lines 19–26
4	myDisplay()	draws the figure, part by part	lines 27–102
5	circleMidpoint()	midpoint circle algorithm (Lec 5)	lines 103–121
6	circleplot()	plots the 8 symmetric points	lines 122–134

1 Headers & prototypes — libraries + declare the functions

START · line 1

```
1  #include <windows.h>
2  #include <GL/glut.h>
3  void myInit();
4  void myDisplay();
5  void circleMidpoint(int xc, int yc, int radius, GLenum mode);
6  void circleplot(int xc, int yc, int x, int y, GLenum mode);
```

END of Headers & prototypes · line 6

2 main() — create the window and start the program

START · line 7

```
7  int main(int argc, char** argv)
8  {
9      glutInit(&argc, argv);
10     glutInitDisplayMode(GLUT_RGB);//set display mode
```

```
11      glutInitWindowSize(400, 300);//set window size
12      glutInitWindowPosition(100, 150);//set window position
13      glutCreateWindow("graphics");
14      myInit();
15      glutDisplayFunc(myDisplay);
16      glutMainLoop();
17      return 0;
18  }
```

END of main() · line 18

3 myInit() background color + coordinate system

START · line 19

```
19  void myInit()
20  {
21      glClearColor(1.0, 1.0, 1.0, 0.0);
22      glColor3f(0.0f, 0.0f, 0.0f);
23      glMatrixMode(GL_PROJECTION);
24      glLoadIdentity();
25      gluOrtho2D(-200, 200, -150, 150);
26  }
```

END of myInit() · line 26

4 myDisplay() draws the figure, part by part

START · line 27

```
27  void myDisplay()
28  {
29      glClear(GL_COLOR_BUFFER_BIT);
30
31      // ground
32      glColor3f(0.4, 0.4, 0.4);
33      glBegin(GL_QUADS);
34      glVertex2f(-200, -150);
35      glVertex2f(200, -150);
36      glVertex2f(200, -120);
37      glVertex2f(-200, -120);
38      glEnd();
39
40      // pole
41      glColor3f(0.5, 0.5, 0.5);
42      glBegin(GL_QUADS);
43      glVertex2f(-10, -120);
44      glVertex2f(10, -120);
45      glVertex2f(10, -20);
46      glVertex2f(-10, -20);
47      glEnd();
48
49      // box
50      glColor3f(0.1, 0.1, 0.1);
51      glBegin(GL_QUADS);
52      glVertex2f(-40, -20);
53      glVertex2f(40, -20);
54      glVertex2f(40, 140);
55      glVertex2f(-40, 140);
56      glEnd();
57
```

```
    // lights
    glColor3f(1.0, 0.0, 0.0);
    circleMidpoint(0, 110, 20, GL_LINES);
    glColor3f(1.0, 0.8, 0.0);
    circleMidpoint(0, 60, 20, GL_LINES);
    glColor3f(0.0, 0.8, 0.0);
    circleMidpoint(0, 10, 20, GL_LINES);

    // zebra crossing
    glColor3f(1.0, 1.0, 1.0);
    glBegin(GL_QUADS);
    glVertex2f(-170, -145);
    glVertex2f(-150, -145);
    glVertex2f(-150, -125);
    glVertex2f(-170, -125);
    glVertex2f(-130, -145);
    glVertex2f(-110, -145);
    glVertex2f(-110, -125);
    glVertex2f(-130, -125);
    glVertex2f(110, -145);
    glVertex2f(130, -145);
    glVertex2f(130, -125);
    glVertex2f(110, -125);
    glVertex2f(150, -145);
    glVertex2f(170, -145);
    glVertex2f(170, -125);
    glVertex2f(150, -125);
    glEnd();

    // sign board
    glColor3f(0.0, 0.3, 0.8);
    glBegin(GL_TRIANGLES);
    glVertex2f(100, 20);
    glVertex2f(160, 20);
    glVertex2f(130, 70);
    glEnd();
    glColor3f(0.5, 0.5, 0.5);
    glLineWidth(4);
    glBegin(GL_LINES);
    glVertex2f(130, 20);
    glVertex2f(130, -120);
    glEnd();

    glFlush();
}
```

END of myDisplay() · line 102

5 **circleMidpoint()** midpoint circle algorithm (Lec 5) START · line 103

```
void circleMidpoint(int xc, int yc, int radius, GLenum mode)
{
    int x = 0;
    int y = radius;
    int p0 = 1 - radius;
    circleplot(xc, yc, x, y, mode);
    while (x < y)
    {
```

```
111         x++;
112         if (p0 < 0)
113             p0 = p0 + 2 * x + 1;
114         else
115         {
116             y--;
117             p0 = p0 + 2 * x -2*y + 1;
118         }
119         circleplot(xc, yc, x, y, mode);
120     }
121 }
```

END of circleMidpoint() · line 121

6 `circleplot()` plots the 8 symmetric points

START · line 122

```
122 void circleplot(int xc, int yc, int x, int y, GLenum mode)
123 {
124     glBegin(mode);
125     glVertex2f(xc + x, yc + y);
126     glVertex2f(xc - x, yc + y);
127     glVertex2f(xc + x, yc - y);
128     glVertex2f(xc - x, yc - y);
129     glVertex2f(xc + y, yc + x);
130     glVertex2f(xc - y, yc + x);
131     glVertex2f(xc + y, yc - x);
132     glVertex2f(xc - y, yc - x);
133     glEnd();
134 }
```

END of circleplot() · line 134

الفكرة البرنامج ملف واحد مقسوم functions: كل function بتبدأ عند START وتخلص عند END. وجوه myDisplay كل جزء من الصورة بيبدأ بـ comment ملوّن.

CG / MODEL ANSWER / MODEL 05

Model Answer — Model 5

M5

Exam paper on page 22 · each answer shows the lecture page of its method

Q1

Source: the page is next to each answer

Question 1 — theory

a · Define

Super-sampling: Increase sampling rate by treating the screen as if it were covered with a finer grid, then use multiple sample points to determine the intensity of each screen pixel. Lec 5 · p25

Reflection: Reflection creates a mirror image of an object across a line or point. Lec 6 · p7

Shear (skewing): Deform shape like shifted slices: shear in x → $x' = x + sh_x \cdot y$, $y' = y$. Lec 6 · p11

b · Compare Cartesian coordinates and display (physical) coordinates Lec 1 · p20

Cartesian	Display
Modeled as floating point	Integer values only
X increasing right, Y increasing up	X increasing right, Y increasing down by 1 unit = 1 pixel

c · True / False

1. ✓ **True** In DDA a line is sampled at unit intervals in one coordinate and the nearest integer values are found for the other. Lec 4 · p15
2. ✗ **False** There is no limit to how big we can make the frame buffer and still keep the refresh rate at 60 frames per second.
   Correct: There is a limit to how big we can make the frame buffer and still maintain the refresh rate at 60 frames or more per second.
   Lec 5 · p24
3. ✓ **True** In a simple color frame buffer each bit plane drives an individual color gun. Lec 2 · p33
4. ✓ **True** With 3 bit planes, one for each primary color, the frame buffer can show 8 colors. Lec 2 · p33
5. ✓ **True** The inverse rotation $R^{-1}(\theta) = R^{T}(\theta)$. Lec 6 · p26

d · Complete

1. The region given by gluOrtho2D(xmin, xmax, ymin, ymax) is known as the _____. → **clipping region** Lec 1 · p21
2. The midpoint of a line segment remains the _____ after a linear transformation. → **midpoint (ratios of distances)** Lec 6 · p3
3. In y = m·x + b: m = _____ and b = _____. → **$(y_{end} - y_o)/(x_{end} - x_o)$ · $y_o - m \cdot x_o$** Lec 4 · p12

e · Explain 1) Why do we use homogeneous coordinates? Lec 6 · p15

- Uniform representation of translation, rotation, scaling.

- Uniform representation of points and vectors.
- Compact representation of a sequence of transformations.
- We add an extra coordinate W (weight = 1) to a point: P(x, y) → P(x, y, 1).

2) Derive the 2D rotation about the origin. Lec 6 · p8–10

- x = r cos φ, y = r sin φ · x′ = r cos(θ + φ), y′ = r sin(θ + φ).
- Substituting gives x′ = x cos θ − y sin θ, y′ = x sin θ + y cos θ.
- R = [cos θ −sin θ; sin θ cos θ], P′ = R · P.

f · Calculate

1. How many gray levels are produced by 2, 4 and 8 bits/pixel?
   → **$2^2 = 4 \cdot 2^4 = 16 \cdot 2^8 = 256$ gray levels** Lec 2 · p23
2. Window (0, 10, 0, 5), viewport (50, 250, 100, 200). A viewport point is (150, 150); find (XW, YW).
   → **Sx = 200/10 = 20, Sy = 100/5 = 20 · 150 = 50 + 20·XW → XW = 5 · 150 = 100 + 20·YW → YW = 2.5 → (5, 2.5)**
   Graphics_Sec 4 · p2

الفكرة الإجابة بجمل المحاضرة نفسها: التعريف جملة، والمقارنة جدول، والـ False لازم يتصحح.

Q2-1

Source: Lec 4 · p19–23 · Graphics sec5 · p1

Question 2 (1) — DDA line · (5, 4) to (12, 13)

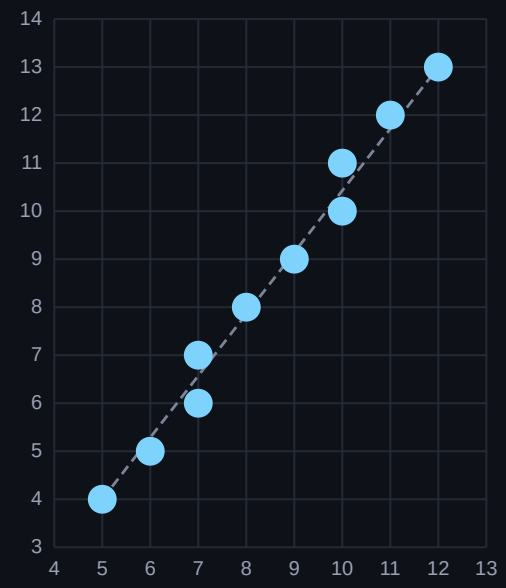
HOW TO SOLVE

1. Compute Δx, Δy, steps and m (or 1/m).
2. Decide the increments (+ or −) from the direction of the line.
3. Add the increments step by step and round each value.
4. Plot the pixels.

Δx = 7, Δy = 9 → |Δy| > |Δx| (|m| > 1) · 1/m = 0.78 · Steps = 9 · $y_{k+1} = y_k + 1$ · $x_{k+1} = x_k + 1/m$

K	x	y	(x, y)
0	5.78	5	(6,5)
1	6.56	6	(7,6)
2	7.33	7	(7,7)
3	8.11	8	(8,8)
4	8.89	9	(9,9)
5	9.67	10	(10,10)
6	10.44	11	(10,11)
7	11.22	12	(11,12)
8	12	13	(12,13)

Plotted pixels



الفكرة الـ coordinate اللي بيتغير أكتر بيمشي بخطوة 1، والتاني بيزيد بالكسر ويتقرّب.

Q2-2

Source: Lec 6 · p20–33 · Lec 1 · p22

Question 2 (2) — P′ = S(2, 2) · T(1, 1) · R(180) · P

Read from right to left — the matrix nearest to P is applied first:

1. **R(180):** Rotate 180 degree around the origin (every point (x, y) goes to (−x, −y)).
2. **T(1, 1):** Translate the object 1 unit to the right and 1 unit up.
3. **S(2, 2):** Scale around the origin: the object becomes double its size — and because it is scaled around the origin, its position is doubled too.

So the object is turned 180° around the origin, moved by (1, 1), then magnified to twice its size about the origin.

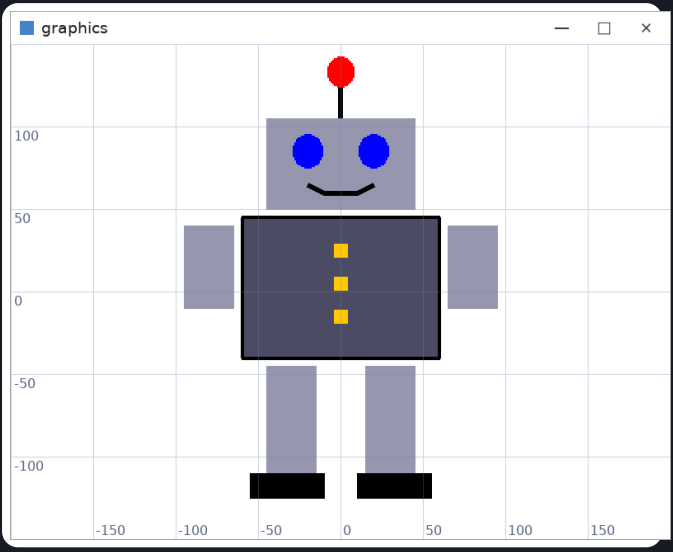
$$\begin{vmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos 180 & -\sin 180 & 0 \\ \sin 180 & \cos 180 & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

الفكرة اقرا الـ matrices من اليمين للشمال، وكل matrix = جملة واحدة بتوصف اللي حصل للشكل.

Q3

Source: main / myInit: Lec 3 · p10 · circleMidpoint: Lec 5 · p14

Question 3 — program



The output with the world-coordinate grid of gluOrtho2D(−200, 200, −150, 150) — read the vertices from it. Any coordinates that give the same figure are correct.

PROGRAM MAP — one file, written in this order

1	**Headers & prototypes**	libraries + declare the functions	lines 1–6
2	**main()**	create the window and start the program	lines 7–18
3	**myInit()**	background color + coordinate system	lines 19–26
4	**myDisplay()**	draws the figure, part by part	lines 27–131
5	**circleMidpoint()**	midpoint circle algorithm (Lec 5)	lines 132–150
6	**circleplot()**	plots the 8 symmetric points	lines 151–163

1 Headers & prototypes — libraries + declare the functions

START · line 1

```
1  #include <windows.h>
2  #include <GL/glut.h>
3  void myInit();
4  void myDisplay();
5  void circleMidpoint(int xc, int yc, int radius, GLenum mode);
6  void circleplot(int xc, int yc, int x, int y, GLenum mode);
```

END of Headers & prototypes · line 6

2 main() — create the window and start the program

START · line 7

```
7  int main(int argc, char** argv)
8  {
9      glutInit(&argc, argv);
10     glutInitDisplayMode(GLUT_RGB);//set display mode
```

```
    glutInitWindowSize(400, 300);//set window size
    glutInitWindowPosition(100, 150);//set window position
    glutCreateWindow("graphics");
    myInit();
    glutDisplayFunc(myDisplay);
    glutMainLoop();
    return 0;
}
```

END of main() · line 18

3 myInit() background color + coordinate system

START · line 19

```
void myInit()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-200, 200, -150, 150);
}
```

END of myInit() · line 26

4 myDisplay() draws the figure, part by part

START · line 27

```
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);

    // antenna
    glColor3f(0.0, 0.0, 0.0);
    glLineWidth(3);
    glBegin(GL_LINES);
    glVertex2f(0, 105);
    glVertex2f(0, 130);
    glEnd();
    glColor3f(1.0, 0.0, 0.0);
    circleMidpoint(0, 133, 8, GL_LINES);

    // head
    glColor3f(0.6, 0.6, 0.7);
    glBegin(GL_QUADS);
    glVertex2f(-45, 50);
    glVertex2f(45, 50);
    glVertex2f(45, 105);
    glVertex2f(-45, 105);
    glEnd();

    // eyes
    glColor3f(0.0, 0.0, 1.0);
    circleMidpoint(-20, 85, 9, GL_LINES);
    circleMidpoint(20, 85, 9, GL_LINES);

    // mouth
    glColor3f(0.0, 0.0, 0.0);
    glLineWidth(3);
```

```
    glBegin(GL_LINE_STRIP);
    glVertex2f(-20, 65);
    glVertex2f(-10, 60);
    glVertex2f(10, 60);
    glVertex2f(20, 65);
    glEnd();

    // body
    glColor3f(0.3, 0.3, 0.4);
    glBegin(GL_QUADS);
    glVertex2f(-60, -40);
    glVertex2f(60, -40);
    glVertex2f(60, 45);
    glVertex2f(-60, 45);
    glEnd();

    // buttons
    glColor3f(1.0, 0.8, 0.0);
    glPointSize(8);
    glBegin(GL_POINTS);
    glVertex2f(0, 25);
    glVertex2f(0, 5);
    glVertex2f(0, -15);
    glEnd();

    // arms
    glColor3f(0.6, 0.6, 0.7);
    glBegin(GL_QUADS);
    glVertex2f(-95, -10);
    glVertex2f(-65, -10);
    glVertex2f(-65, 40);
    glVertex2f(-95, 40);
    glVertex2f(65, -10);
    glVertex2f(95, -10);
    glVertex2f(95, 40);
    glVertex2f(65, 40);
    glEnd();

    // legs
    glBegin(GL_QUADS);
    glVertex2f(-45, -110);
    glVertex2f(-15, -110);
    glVertex2f(-15, -45);
    glVertex2f(-45, -45);
    glVertex2f(15, -110);
    glVertex2f(45, -110);
    glVertex2f(45, -45);
    glVertex2f(15, -45);
    glEnd();

    // feet
    glColor3f(0.0, 0.0, 0.0);
    glBegin(GL_QUADS);
    glVertex2f(-55, -125);
    glVertex2f(-10, -125);
    glVertex2f(-10, -110);
    glVertex2f(-55, -110);
    glVertex2f(10, -125);
    glVertex2f(55, -125);
```

```
117     glVertex2f(55, -110);
118     glVertex2f(10, -110);
119     glEnd();
120
121     // outline of the body
122     glLineWidth(2);
123     glBegin(GL_LINE_LOOP);
124     glVertex2f(-60, -40);
125     glVertex2f(60, -40);
126     glVertex2f(60, 45);
127     glVertex2f(-60, 45);
128     glEnd();
129
130     glFlush();
131 }
```

END of myDisplay() · line 131

5 circleMidpoint() midpoint circle algorithm (Lec 5)

START · line 132

```
132 void circleMidpoint(int xc, int yc, int radius, GLenum mode)
133 {
134     int x = 0;
135     int y = radius;
136     int p0 = 1 - radius;
137     circleplot(xc, yc, x, y, mode);
138     while (x < y)
139     {
140         x++;
141         if (p0 < 0)
142             p0 = p0 + 2 * x + 1;
143         else
144         {
145             y--;
146             p0 = p0 + 2 * x -2*y + 1;
147         }
148         circleplot(xc, yc, x, y, mode);
149     }
150 }
```

END of circleMidpoint() · line 150

6 circleplot() plots the 8 symmetric points

START · line 151

```
151 void circleplot(int xc, int yc, int x, int y, GLenum mode)
152 {
153     glBegin(mode);
154     glVertex2f(xc + x, yc + y);
155     glVertex2f(xc - x, yc + y);
156     glVertex2f(xc + x, yc - y);
157     glVertex2f(xc - x, yc - y);
158     glVertex2f(xc + y, yc + x);
159     glVertex2f(xc - y, yc + x);
160     glVertex2f(xc + y, yc - x);
161     glVertex2f(xc - y, yc - x);
```

```
162     glEnd();
163 }
```

END of circleplot() · line 163

الفكرة البرنامج ملف واحد مقسوم functions: كل function بتبدأ عند START وتخلص عند END. وجوه myDisplay كل جزء من الصورة بيبدأ بـ comment ملوّن.

CG / MODEL ANSWER / MODEL 06

Model Answer — Model 6

M6

Exam paper on page 24 · each answer shows the lecture page of its method

Q1

Source: the page is next to each answer

Question 1 — theory

a · Define

Image processing: Analyze pictures to derive descriptions (often in mathematical or geometrical forms) of objects appeared in the pictures. Lec 1 · p6

Extent (bounding box): The aligned rectangle that just covers the object. Lec 3 · p21

Valuator: Input primitive: scalar values (typically between 0 and 1); can be used to fix the length of a line, the speed of an action, or the size of a picture. Lec 1 · p13

b · Compare Frame rate and refresh rate Lec 2 · p28

Frame rate (FPS)	Refresh rate (Hz)
How many complete images the GPU can create and send to the monitor each second	How many times per second the monitor can physically update the image
Variable — depends on the power of the device	Fixed maximum — determined by the monitor's hardware (e.g. 60 Hz, 144 Hz)

c · True / False

1. ✗ **False** The primary interface of OpenGL is in Java.
   Correct: OpenGL supports many languages (C++, Python, Java, …) but the primary interface of OpenGL is in C++.
   Lec 1 · p19
2. ✓ **True** 4 bits/pixel produce 16 gray levels. Lec 2 · p23
3. ✗ **False** Two translations combine by multiplying their values.
   Correct: T(tx1, ty1) · T(tx2, ty2) = T(tx1 + tx2, ty1 + ty2) — translations add.
   Lec 6 · p20–22
4. ✓ **True** In 2D there is no need to simulate lights, cameras, etc. Lec 1 · p18
5. ✗ **False** In Cartesian form translation is a multiplication $P' = T \cdot P$.
   Correct: Translation is $P' = T + P$; we would like all transformations to be multiplications, so we use homogeneous coordinates.
   Lec 6 · p13–15

d · Complete

1. Attributes of lines and polylines are _____, _____, _____ and the way edges are blended. → **color · thickness · type (dashed, solid, dotted)** Lec 2 · p14
2. Rotation property: R(θ) · R(φ) = _____. → **R(θ + φ)** Lec 6 · p26

3. If the frame rate and the refresh rate are not equal, the _____ one will put a cap on the higher one. → **lesser** Lec 2 · p28

e · Explain 1) Explain the role of the CPU, the GPU and the frame buffer. Lec 1 · p12

- **CPU:** gets the specifications for a geometric shape from the input device and hands the geometric data to the GPU.
- **GPU:** rasterizes the data by deciding which pixels to use, then places the pixel data into the framebuffer.
- **Frame buffer:** a block of memory that holds data representing all the pixels in a complete video frame.

2) What are the three basic elements? Define each. Lec 4 · p3

- **Points:** associated with locations in space; only their position distinguishes them.
- **Lines:** a one-dimensional shape with a fixed position in 3D space.
- **Vectors:** have magnitude and direction.

f · Calculate

1. System A: 1920 × 1080 display, 32 bits/pixel, no LUT. Find the frame buffer memory.
   → **1920 × 1080 × 32 = 66,355,200 bits / 8 = 8,294,400 bytes ≈ 8.29 MB** Graphics_SEC 3 · p2
2. Midpoint circle with r = 10: find P0 and the first three points.
   → **P0 = 1 − 10 = −9 → (1, 10) · P = −6 → (2, 10) · P = −1 → (3, 10)** Lec 5 · p12

الفكرة الإجابة بجمل المحاضرة نفسها: التعريف جملة، والمقارنة جدول، والـ False لازم يتصحح.

Q2-1 Source: Lec 5 · p16–23 · Graphics_Sec 6 · p2

Question 2 (1) — Midpoint ellipse · $r_x = 8$ and $r_y = 4$

HOW TO SOLVE

1. Compute r_x^2, r_y^2 and $p1_0$.
2. Region 1: fill the table (increment x) until $2r_y^2x \geq 2r_x^2y$.
3. Compute $p2_0$ from the last point of region 1.
4. Region 2: fill the table (decrement y) until y = 0.
5. Plot the points.

$r_x^2 = 64$, $r_y^2 = 16$ · initial point (0, 4)

$p1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4}r_x^2 = 16 - 64\times4 + \frac{1}{4}\times64 = -224$

Region 1

Pk_region1	x	y	$2r_y^2x$	$2r_x^2y$
	0	4	0	512
−224	1	4	32	512
−176	2	4	64	512
−96	3	4	96	512

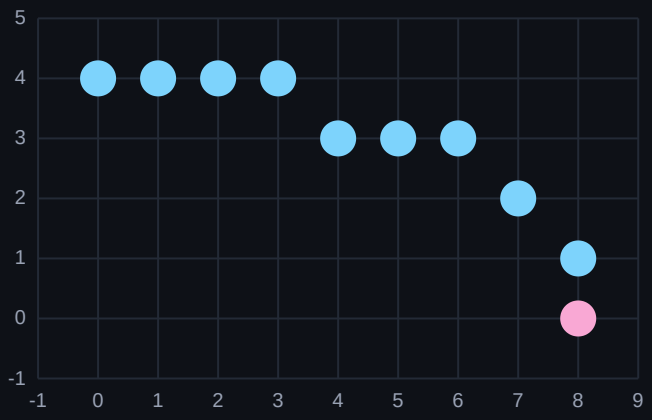
16	4	3	128	384
−224	5	3	160	384
−48	6	3	192	384
160	7	2	224	256
144	8	1	256	128

Region 1 stops when $2r_y^2x \geq 2r_x^2y$ → at (8, 1): 256 ≥ 128
Initial point of region 2 = last point of region 1 (8, 1) · $p2_0 = r_y^2(x_0 + \frac{1}{2})^2 + r_x^2(y_0 - 1)^2 - r_x^2r_y^2 = 16\times(8.5)^2 + 64\times(0)^2 - 64\times16 = 132$

Region 2

Pk_region2	x	y	$2r_y^2x$	$2r_x^2y$
	8	1	256	128
132	8	0		

Region 1 (blue) · Region 2 (pink)



الفكرة Region 1 بنزوّد x لحد ما $x_y^2 2r$ يبقى $\leq y_x^2 2r$، وبعدين Region 2 بنقلّل y لحد الصفر.

Q2-2

Source: Lec 6 · p20–33 · Lec 1 · p22

Question 2 (2) — P′ = T(5, 5) · S(2, 2) · R(90) · T(−2, −1) · P

Read from right to left — the matrix nearest to P is applied first:

1. **T(−2, −1):** Translate the object so that the point (2, 1) moves to the origin.
2. **R(90):** Rotate 90 degree counter-clockwise around the origin.
3. **S(2, 2):** Scale around the origin: the object becomes double its size.
4. **T(5, 5):** Translate the object so that the point that was at (2, 1) ends at (5, 5).

So the object is rotated 90° and magnified 2 times around the point (2, 1), then that point is moved to (5, 5).

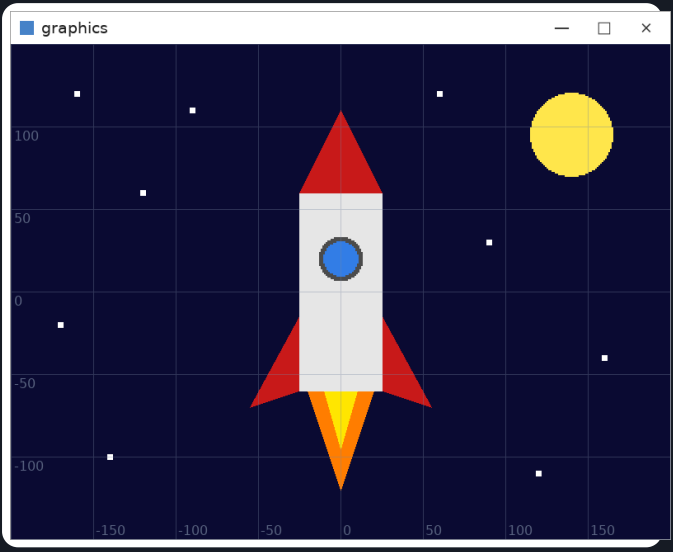
$$\begin{vmatrix} 1 & 0 & 5 \\ 0 & 1 & 5 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos 90 & -\sin 90 & 0 \\ \sin 90 & \cos 90 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & -2 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{vmatrix}$$

الفكرة اقرا الـ matrices من اليمين للشمال، وكل matrix = جملة واحدة بتوصف اللي حصل للشكل.

Q3

Source: main / myInit: Lec 3 · p10 · circleMidpoint: Lec 5 · p14

Question 3 — program



The output with the world-coordinate grid of gluOrtho2D(−200, 200, −150, 150) — read the vertices from it. Any coordinates that give the same figure are correct.

PROGRAM MAP — one file, written in this order

1	**Headers & prototypes**	libraries + declare the functions	lines 1–6
2	**main()**	create the window and start the program	lines 7–18
3	**myInit()**	background color + coordinate system	lines 19–26
4	**myDisplay()**	draws the figure, part by part	lines 27–109
5	**circleMidpoint()**	midpoint circle algorithm (Lec 5)	lines 110–128
6	**circleplot()**	plots the 8 symmetric points	lines 129–141

1 Headers & prototypes — libraries + declare the functions

START · line 1

```
1  #include <windows.h>
2  #include <GL/glut.h>
3  void myInit();
4  void myDisplay();
5  void circleMidpoint(int xc, int yc, int radius, GLenum mode);
6  void circleplot(int xc, int yc, int x, int y, GLenum mode);
```

END of Headers & prototypes · line 6

2 main() — create the window and start the program

START · line 7

```
7  int main(int argc, char** argv)
8  {
9      glutInit(&argc, argv);
10     glutInitDisplayMode(GLUT_RGB);//set display mode
```

```
11      glutInitWindowSize(400, 300);//set window size
12      glutInitWindowPosition(100, 150);//set window position
13      glutCreateWindow("graphics");
14      myInit();
15      glutDisplayFunc(myDisplay);
16      glutMainLoop();
17      return 0;
18  }
```

END of main() · line 18

3 myInit() background color + coordinate system

START · line 19

```
19  void myInit()
20  {
21      glClearColor(1.0, 1.0, 1.0, 0.0);
22      glColor3f(0.0f, 0.0f, 0.0f);
23      glMatrixMode(GL_PROJECTION);
24      glLoadIdentity();
25      gluOrtho2D(-200, 200, -150, 150);
26  }
```

END of myInit() · line 26

4 myDisplay() draws the figure, part by part

START · line 27

```
27  void myDisplay()
28  {
29      glClear(GL_COLOR_BUFFER_BIT);
30
31      // night sky
32      glColor3f(0.1, 0.1, 0.2);
33      glBegin(GL_QUADS);
34      glVertex2f(-200, -150);
35      glVertex2f(200, -150);
36      glVertex2f(200, 150);
37      glVertex2f(-200, 150);
38      glEnd();
39
40      // stars
41      glColor3f(1.0, 1.0, 1.0);
42      glPointSize(3);
43      glBegin(GL_POINTS);
44      glVertex2f(-160, 120);
45      glVertex2f(-120, 60);
46      glVertex2f(-170, -20);
47      glVertex2f(-90, 110);
48      glVertex2f(90, 30);
49      glVertex2f(160, -40);
50      glVertex2f(120, -110);
51      glVertex2f(-140, -100);
52      glVertex2f(60, 120);
53      glEnd();
54
55      // moon
56      glColor3f(1.0, 0.9, 0.3);
57      circleMidpoint(140, 95, 25, GL_LINES);
```

```
58
59      // flame
60      glColor3f(1.0, 0.5, 0.0);
61      glBegin(GL_TRIANGLES);
62      glVertex2f(-20, -60);
63      glVertex2f(20, -60);
64      glVertex2f(0, -120);
65      glEnd();
66      glColor3f(1.0, 0.9, 0.0);
67      glBegin(GL_TRIANGLES);
68      glVertex2f(-10, -60);
69      glVertex2f(10, -60);
70      glVertex2f(0, -95);
71      glEnd();
72
73      // fins
74      glColor3f(0.8, 0.1, 0.1);
75      glBegin(GL_TRIANGLES);
76      glVertex2f(-25, -60);
77      glVertex2f(-25, -15);
78      glVertex2f(-55, -70);
79      glVertex2f(25, -60);
80      glVertex2f(25, -15);
81      glVertex2f(55, -70);
82      glEnd();
83
84      // body
85      glColor3f(0.9, 0.9, 0.9);
86      glBegin(GL_QUADS);
87      glVertex2f(-25, -60);
88      glVertex2f(25, -60);
89      glVertex2f(25, 60);
90      glVertex2f(-25, 60);
91      glEnd();
92
93      // nose
94      glColor3f(0.8, 0.1, 0.1);
95      glBegin(GL_TRIANGLES);
96      glVertex2f(-25, 60);
97      glVertex2f(25, 60);
98      glVertex2f(0, 110);
99      glEnd();
100
101     // window
102     glColor3f(0.2, 0.5, 0.9);
103     circleMidpoint(0, 20, 12, GL_LINES);
104     glColor3f(0.3, 0.3, 0.3);
105     glPointSize(2);
106     circleMidpoint(0, 20, 12, GL_POINTS);
107
108     glFlush();
109 }
```

END of myDisplay() · line 109

5 `circleMidpoint()` midpoint circle algorithm (Lec 5)

START · line 110

```
110 void circleMidpoint(int xc, int yc, int radius, GLenum mode)
111 {
112     int x = 0;
113     int y = radius;
114     int p0 = 1 - radius;
115     circleplot(xc, yc, x, y, mode);
116     while (x < y)
117     {
118         x++;
119         if (p0 < 0)
120             p0 = p0 + 2 * x + 1;
121         else
122         {
123             y--;
124             p0 = p0 + 2 * x -2*y + 1;
125         }
126         circleplot(xc, yc, x, y, mode);
127     }
128 }
```

END of circleMidpoint() · line 128

6 `circleplot()` plots the 8 symmetric points

START · line 129

```
129 void circleplot(int xc, int yc, int x, int y, GLenum mode)
130 {
131     glBegin(mode);
132     glVertex2f(xc + x, yc + y);
133     glVertex2f(xc - x, yc + y);
134     glVertex2f(xc + x, yc - y);
135     glVertex2f(xc - x, yc - y);
136     glVertex2f(xc + y, yc + x);
137     glVertex2f(xc - y, yc + x);
138     glVertex2f(xc + y, yc - x);
139     glVertex2f(xc - y, yc - x);
140     glEnd();
141 }
```

END of circleplot() · line 141

الفكرة البرنامج ملف واحد مقسوم functions: كل function بتبدأ عند START وتخلص عند END. وجوه myDisplay كل جزء من الصورة بيبدأ بـ comment ملوّن.

CG / MODEL ANSWER / MODEL 07

Model Answer — Model 7

M7

Exam paper on page 26 · each answer shows the lecture page of its method

Q1

Source: the page is next to each answer

Question 1 — theory

a · Define

Sampling: Measuring the values of the function at equal intervals. Lec 4 · p8

Monochrome raster image: Pixels can contain one of two values in a raster image (bi-level). Lec 2 · p22

Output device: Displays data from memory of the computer, e.g. CRT, LED, LCD, plasma display, plotter, hologram generator. SEC 1 and 2 · p1

b · Compare Simple, complex, concave and convex polygons Lec 2 · p12 · Graphics_SEC 3 · p1

Polygon	Meaning
Simple	Has well defined two regions (inside and outside)
Complex	Has self-intersecting sides
Concave	At least one diagonal lies outside the polygon (curved inward)
Convex	All its diagonals lie inside the polygon (curved outward)

c · True / False

1. ✗ **False** The midpoint circle algorithm must calculate the points of all eight octants.
   Correct: Only one octant's calculation is needed; the other seven come from the eight-way symmetry.
   Lec 5 · p10, p13
2. ✓ **True** The software level contains the application program, the application model and the graphics library.
   Lec 1 · p17
3. ✗ **False** glutInitWindowPosition gives the location of the lower-left corner of the display window.
   Correct: It gives an initial location for the upper left corner of the display window.
   Lec 1 · p21
4. ✓ **True** 2-D vector output devices physically draw line segments (like a plotter). SEC 1 and 2 · p1
5. ✗ **False** Shear in y: $x' = x + sh_y \cdot y$.
   Correct: Shear in y: $x' = x$, $y' = sh_y \cdot x + y$.
   Lec 6 · p11

d · Complete

1. We use _____ to model light and to perform simulations for animation, and _____ to describe shape and motion. → **physics · mathematics** Lec 1 · p7

2. Supersampling gives more weight to subpixels near the _____ of a pixel area. → **center** Lec 5 · p25
3. In a raster image each numerical value of the 2D array represents the _____ stored there. → **value of the pixel** Lec 2 · p19

e · Explain 1) Why does the simple approach (increment x, solve for y) fail for lines with slope greater than 1? Lec 4 · p13

- It works for lines with a slope of 1 or less.
- For slope greater than 1, lines become more discontinuous in appearance and we must add more than 1 pixel per column.
- Solution: use symmetry (sample in y instead).

2) How can the application set the window automatically? Lec 3 · p21

- Often the programmer does not know where or how big the object lies in world coordinates.
- Find a window that includes the entire object: the object's extent must be found.
- The extent (bounding box) is the aligned rectangle that just covers the object.

f · Calculate

1. Find the matching viewport for a window with aspect ratio R = 2 when the screen window is 360 × 200.
   → **W/H = 1.8 < R → width = W = 360, height = W/R = 180 → glViewport(0, 0, 360, 180)** Lec 3 · p27
2. For an ellipse with $r_x = 8$ and $r_y = 6$ find the initial point and $p1_0$.
   → **(0, 6) · $p1_0 = 36 - 64\times6 + \frac{1}{4}\times64 = -332$** Lec 5 · p22

الفكرة الإجابة بجمل المحاضرة نفسها: التعريف جملة، والمقارنة جدول، والـ False لازم يتصحح.

Q2-1

Source: Lec 5 · p11–13 · Graphics_Sec 6 · p1

Question 2 (1) — Midpoint circle · r = 11

HOW TO SOLVE

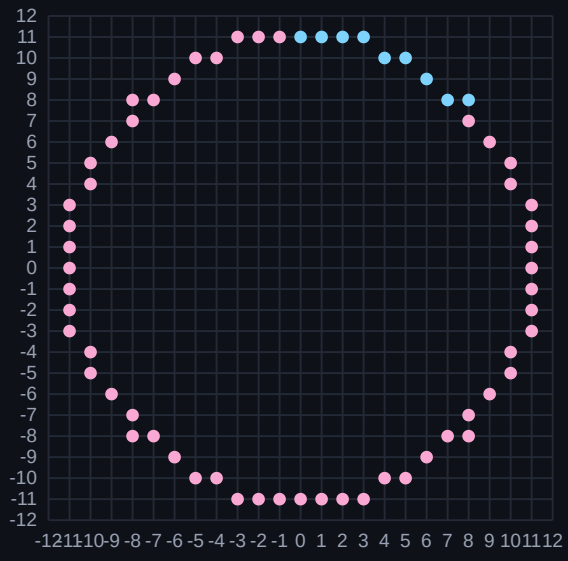
1. P0 = 1 − r, start at (0, r).
2. Use the sign of P to choose y or y − 1, then update P.
3. Stop when x ≥ y.
4. Plot the octant and its symmetry.

Initial point (0, 11) · P0 = 1 − r = -10

P	X	Y	$2x_{k+1}$	$2y_{k+1}$
-	0	11	-	-
−10	1	11	2	22
−7	2	11	4	22
−2	3	11	6	22
5	4	10	8	20

Octant points (blue) + eight-way symmetry (pink)

−6	5	10	10	20
5	6	9	12	18
0	7	8	14	16
−1	8	8	-	-



الفكرة إشارة P بتحدد y هيفضل ثابت ولا هينزل 1، وبنقف لما x يوصل لـ y.

Q2-2

Source: Lec 6 · p20–33 · Lec 1 · p22

Question 2 (2) — P′ = T(4, −1) · S(1, 2) · R(−90) · P

Read from right to left — the matrix nearest to P is applied first:

1. **R(−90):** Rotate 90 degree clockwise around the origin (negative angle).
2. **S(1, 2):** Scale around the origin: y is multiplied by 2 (stretched 2 times vertically), x does not change.
3. **T(4, −1):** Translate the object 4 units to the right and 1 unit down.

So the object is turned 90° clockwise about the origin, stretched 2 times vertically, then moved by (4, −1).

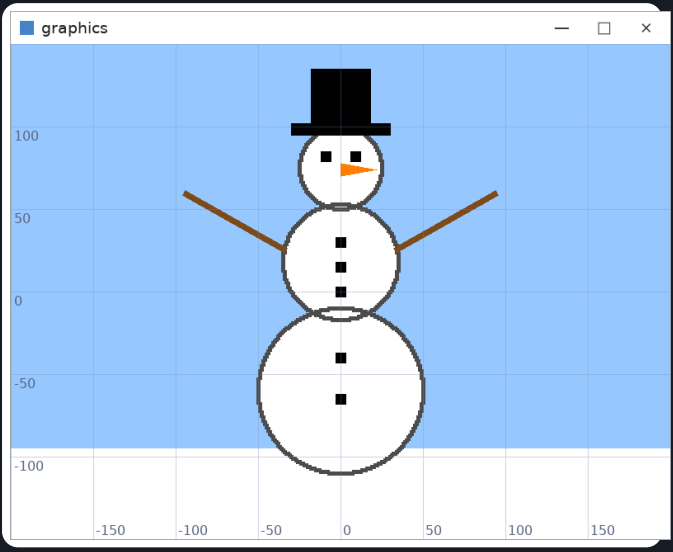
$$\begin{vmatrix} 1 & 0 & 4 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos -90 & -\sin -90 & 0 \\ \sin -90 & \cos -90 & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

الفكرة اقرا الـ matrices من اليمين للشمال، وكل matrix = جملة واحدة بتوصف اللي حصل للشكل.

Q3

Source: main / myInit: Lec 3 · p10 · circleMidpoint: Lec 5 · p14

Question 3 — program



The output with the world-coordinate grid of gluOrtho2D(−200, 200, −150, 150) — read the vertices from it. Any coordinates that give the same figure are correct.

PROGRAM MAP — one file, written in this order

1	**Headers & prototypes**	libraries + declare the functions	lines 1–6
2	**main()**	create the window and start the program	lines 7–18
3	**myInit()**	background color + coordinate system	lines 19–26
4	**myDisplay()**	draws the figure, part by part	lines 27–104
5	**circleMidpoint()**	midpoint circle algorithm (Lec 5)	lines 105–123
6	**circleplot()**	plots the 8 symmetric points	lines 124–136

1 Headers & prototypes — libraries + declare the functions

START · line 1

```
1  #include <windows.h>
2  #include <GL/glut.h>
3  void myInit();
4  void myDisplay();
5  void circleMidpoint(int xc, int yc, int radius, GLenum mode);
6  void circleplot(int xc, int yc, int x, int y, GLenum mode);
```

END of Headers & prototypes · line 6

2 main() — create the window and start the program

START · line 7

```
7  int main(int argc, char** argv)
8  {
9      glutInit(&argc, argv);
10     glutInitDisplayMode(GLUT_RGB);//set display mode
```

```
11      glutInitWindowSize(400, 300);//set window size
12      glutInitWindowPosition(100, 150);//set window position
13      glutCreateWindow("graphics");
14      myInit();
15      glutDisplayFunc(myDisplay);
16      glutMainLoop();
17      return 0;
18  }
```

END of main() · line 18

3 myInit() background color + coordinate system

START · line 19

```
19  void myInit()
20  {
21      glClearColor(1.0, 1.0, 1.0, 0.0);
22      glColor3f(0.0f, 0.0f, 0.0f);
23      glMatrixMode(GL_PROJECTION);
24      glLoadIdentity();
25      gluOrtho2D(-200, 200, -150, 150);
26  }
```

END of myInit() · line 26

4 myDisplay() draws the figure, part by part

START · line 27

```
27  void myDisplay()
28  {
29      glClear(GL_COLOR_BUFFER_BIT);
30
31      // sky
32      glColor3f(0.6, 0.8, 1.0);
33      glBegin(GL_QUADS);
34      glVertex2f(-200, -150);
35      glVertex2f(200, -150);
36      glVertex2f(200, 150);
37      glVertex2f(-200, 150);
38      glEnd();
39
40      // snow
41      glColor3f(1.0, 1.0, 1.0);
42      glBegin(GL_QUADS);
43      glVertex2f(-200, -150);
44      glVertex2f(200, -150);
45      glVertex2f(200, -95);
46      glVertex2f(-200, -95);
47      glEnd();
48
49      // body
50      circleMidpoint(0, -60, 50, GL_LINES);
51      circleMidpoint(0, 18, 35, GL_LINES);
52      circleMidpoint(0, 75, 25, GL_LINES);
53      glColor3f(0.3, 0.3, 0.3);
54      glPointSize(2);
55      circleMidpoint(0, -60, 50, GL_POINTS);
56      circleMidpoint(0, 18, 35, GL_POINTS);
57      circleMidpoint(0, 75, 25, GL_POINTS);
```

```
58
59      // eyes and buttons
60      glColor3f(0.0, 0.0, 0.0);
61      glPointSize(6);
62      glBegin(GL_POINTS);
63      glVertex2f(-9, 82);
64      glVertex2f(9, 82);
65      glVertex2f(0, 30);
66      glVertex2f(0, 15);
67      glVertex2f(0, 0);
68      glVertex2f(0, -40);
69      glVertex2f(0, -65);
70      glEnd();
71
72      // nose
73      glColor3f(1.0, 0.5, 0.0);
74      glBegin(GL_TRIANGLES);
75      glVertex2f(0, 78);
76      glVertex2f(0, 70);
77      glVertex2f(22, 74);
78      glEnd();
79
80      // hat
81      glColor3f(0.0, 0.0, 0.0);
82      glBegin(GL_QUADS);
83      glVertex2f(-30, 95);
84      glVertex2f(30, 95);
85      glVertex2f(30, 102);
86      glVertex2f(-30, 102);
87      glVertex2f(-18, 102);
88      glVertex2f(18, 102);
89      glVertex2f(18, 135);
90      glVertex2f(-18, 135);
91      glEnd();
92
93      // arms
94      glColor3f(0.5, 0.3, 0.1);
95      glLineWidth(4);
96      glBegin(GL_LINES);
97      glVertex2f(-33, 25);
98      glVertex2f(-95, 60);
99      glVertex2f(33, 25);
100     glVertex2f(95, 60);
101     glEnd();
102
103     glFlush();
104 }
```

END of myDisplay() · line 104

5 **circleMidpoint()** midpoint circle algorithm (Lec 5) START · line 105

```
105 void circleMidpoint(int xc, int yc, int radius, GLenum mode)
106 {
107     int x = 0;
108     int y = radius;
109     int p0 = 1 - radius;
110     circleplot(xc, yc, x, y, mode);
```

```
111     while (x < y)
112     {
113         x++;
114         if (p0 < 0)
115             p0 = p0 + 2 * x + 1;
116         else
117         {
118             y--;
119             p0 = p0 + 2 * x -2*y + 1;
120         }
121         circleplot(xc, yc, x, y, mode);
122     }
123 }
```

END of circleMidpoint() · line 123

6 `circleplot()` plots the 8 symmetric points

START · line 124

```
124 void circleplot(int xc, int yc, int x, int y, GLenum mode)
125 {
126     glBegin(mode);
127     glVertex2f(xc + x, yc + y);
128     glVertex2f(xc - x, yc + y);
129     glVertex2f(xc + x, yc - y);
130     glVertex2f(xc - x, yc - y);
131     glVertex2f(xc + y, yc + x);
132     glVertex2f(xc - y, yc + x);
133     glVertex2f(xc + y, yc - x);
134     glVertex2f(xc - y, yc - x);
135     glEnd();
136 }
```

END of circleplot() · line 136

الفكرة البرنامج ملف واحد مقسوم functions: كل function بتبدأ عند START وتخلص عند END. وجوه myDisplay كل جزء من الصورة بيبدأ بـ comment ملوّن.

CG / MODEL ANSWER / MODEL 08

Model Answer — Model 8

M8

Exam paper on page 28 · each answer shows the lecture page of its method

Q1

Source: the page is next to each answer

Question 1 — theory

a · Define

Linear transformation: A transformation that preserves co-linearity and ratios of distances (parallel lines remain parallel); represented by a non-singular square matrix. Lec 6 · p3, p5

Raster display: A display that uses a frame buffer to store the color value for each screen position (pixel). Lec 1 · p15

Translation: Translate over vector (t_x, t_y): $x' = x + t_x$, $y' = y + t_y$ ($P' = P + T$). Lec 6 · p12

b · Compare DDA and Bresenham line algorithms Lec 4 · p24–28

DDA	Bresenham
Floating-point arithmetic (time-consuming)	Uses only incremental integer calculations
Round-off error accumulates and pixels drift from the true path	Accurate and efficient
Simple algorithm to implement	Uses a decision parameter p to choose y or y + 1

c · True / False

1. ✗ **False** The viewport is what to be displayed.
   Correct: The window is what to be displayed; the viewport is where to be displayed.
   Lec 3 · p3
2. ✓ **True** To ensure device independence, graphics systems convert object descriptions into normalized coordinates. Lec 3 · p2
3. ✓ **True** In scaling, the object must be centered on the origin, or it will be shifted. Lec 6 · p6
4. ✓ **True** Rotation in 3D needs to specify which axis the rotation is about. Lec 6 · p40
5. ✗ **False** $S(s_{x1}, s_{y1}) \cdot S(s_{x2}, s_{y2}) = S(s_{x1} + s_{x2}, s_{y1} + s_{y2})$.
   Correct: Scales multiply: $S(s_{x1} \cdot s_{x2}, s_{y1} \cdot s_{y2})$.
   Lec 6 · p24

d · Complete

1. Vector graphics is known as _____. → **scalable vector graphics (SVG)** Lec 1 · p15
2. A frame buffer that supports true color has _____ planes of memory, _____ planes for each color. → **24 · 8** Lec 2 · p35
3. In Bresenham, for $x_{k+1} = x_k + 1$ we choose between _____ and _____. → **$(x_k + 1, y_k)$ · $(x_k + 1, y_k + 1)$** Lec 4 · p27

e · Explain 1) List the input devices and their input primitives. Lec 1 · p13–14

- **Locator** (interactive drawing): coordinate position (x, y).
- **String** (text input): a string of characters.
- **Valuator** (games): scalar values (typically between 0 and 1).
- **Choice** (interactive menu selection): a selection from a list of options.
- **Pick**: selection of a part of the screen.

2) What is a 2D composite transformation and when is it used? Lec 6 · p29

- Combine transformations of different type: translate–rotate–translate, translate–scale–translate, translate–reflect–translate.
- Used to rotate or scale an object w.r.t. a point that is not the origin.
- Implemented by multiplication of the corresponding homogeneous matrices.

f · Calculate

1. A frame buffer supports 6 planes of memory (2 planes for each color). How many colors are in the palette?
   → **Each DAC has 2 input bits → $2^2 = 4$ levels → $4 \times 4 \times 4 = 2^6 = 64$ colors** Lec 2 · p35
2. Rotate the point B(2, 1) by 90° about the origin.
   → **$x' = 2 \cos 90 - 1 \sin 90 = -1 \cdot y' = 2 \sin 90 + 1 \cos 90 = 2$ → $B'(-1, 2)$** Lec 6 · p10, p34

الفكرة الإجابة بجمل المحاضرة نفسها: التعريف جملة، والمقارنة جدول، والـ False لازم يتصحح.

Q2-1 Source: Lec 4 · p28–30 · Graphics sec5 · p2

Question 2 (1) — Bresenham line · (30, 18) and (42, 25)

HOW TO SOLVE

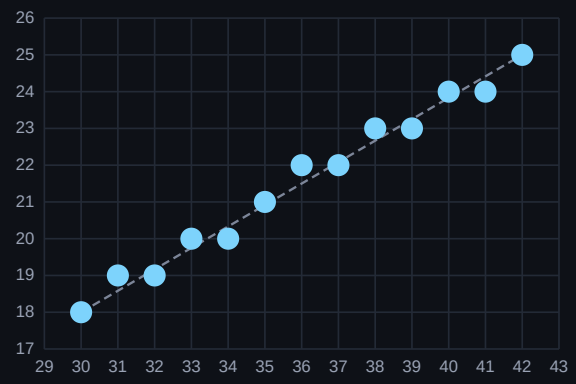
1. Compute Δx, Δy and p0 = 2Δy − Δx.
2. Find the two update values: + 2Δy and + 2Δy − 2Δx.
3. Fill the table: the sign of P decides if y increases.
4. Plot the pixels.

$|\Delta x| = |42 - 30| = 12$, $|\Delta y| = |25 - 18| = 7$ · $|\Delta x| > |\Delta y|$ · $p0 = 2\Delta y - \Delta x = 14 - 12 = 2$ · $P_k < 0$: x incremented, y remains, $P_{k+1} = P_k + 14$ · $P_k \geq 0$: x and y incremented, $P_{k+1} = P_k - 10$

P	X	Y	SetPixel(x, y)
-	30	18	(30,18)
2	31	19	(31,19)
−8	32	19	(32,19)
6	33	20	(33,20)
−4	34	20	(34,20)
10	35	21	(35,21)

Plotted pixels

0	36	22	(36,22)
−10	37	22	(37,22)
4	38	23	(38,23)
−6	39	23	(39,23)
8	40	24	(40,24)
−2	41	24	(41,24)
12	42	25	(42,25)



الفكرة الـ P السالب يخلّي y ثابت، والموجب (أو الصفر) يزوّد y بواحد.

Q2-2

Source: Lec 6 · p20–33 · Lec 1 · p22

Question 2 (2) — P′ = T(2, 6) · S(−1, 1) · T(−2, −6) · P

Read from right to left — the matrix nearest to P is applied first:

1. **T(−2, −6):** Translate the object so that the point (2, 6) moves to the origin.
2. **S(−1, 1):** Scale with $s_x = -1$, $s_y = 1$: x → −x, y stays — a mirror image (reflection) across the y-axis.
3. **T(2, 6):** Translate the object so that the point (2, 6) is back to its original position.

This is "translate, reflect, translate": the object is mirrored across the vertical line x = 2.

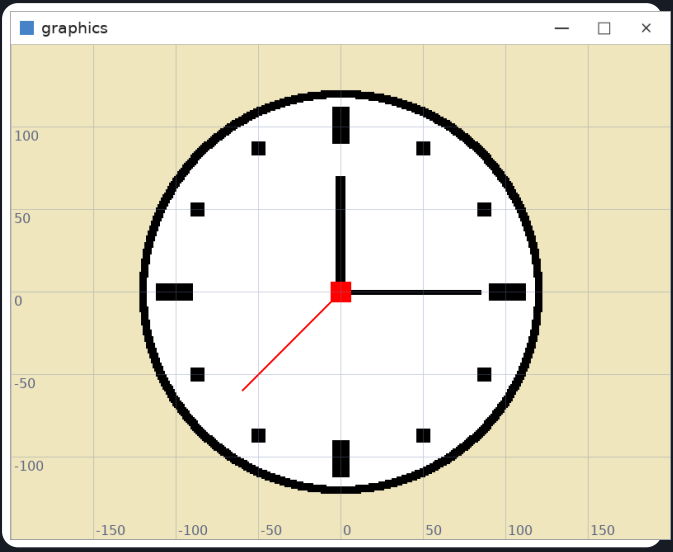
$$\begin{vmatrix} 1 & 0 & 2 \\ 0 & 1 & 6 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & -2 \\ 0 & 1 & -6 \\ 0 & 0 & 1 \end{vmatrix}$$

الفكرة اقرا الـ matrices من اليمين للشمال، وكل matrix = جملة واحدة بتوصف اللي حصل للشكل.

Q3

Source: main / myInit: Lec 3 · p10 · circleMidpoint: Lec 5 · p14

Question 3 — program



The output with the world-coordinate grid of gluOrtho2D(−200, 200, −150, 150) — read the vertices from it. Any coordinates that give the same figure are correct.

PROGRAM MAP — one file, written in this order

1	**Headers & prototypes**	libraries + declare the functions	lines 1–6
2	**main()**	create the window and start the program	lines 7–18
3	**myInit()**	background color + coordinate system	lines 19–26
4	**myDisplay()**	draws the figure, part by part	lines 27–103
5	**circleMidpoint()**	midpoint circle algorithm (Lec 5)	lines 104–122
6	**circleplot()**	plots the 8 symmetric points	lines 123–135

1 Headers & prototypes — libraries + declare the functions

START · line 1

```
1  #include <windows.h>
2  #include <GL/glut.h>
3  void myInit();
4  void myDisplay();
5  void circleMidpoint(int xc, int yc, int radius, GLenum mode);
6  void circleplot(int xc, int yc, int x, int y, GLenum mode);
```

END of Headers & prototypes · line 6

2 main() — create the window and start the program

START · line 7

```
7  int main(int argc, char** argv)
8  {
9      glutInit(&argc, argv);
10     glutInitDisplayMode(GLUT_RGB);//set display mode
```

```
    glutInitWindowSize(400, 300);//set window size
    glutInitWindowPosition(100, 150);//set window position
    glutCreateWindow("graphics");
    myInit();
    glutDisplayFunc(myDisplay);
    glutMainLoop();
    return 0;
}
```

END of main() · line 18

3 myInit() background color + coordinate system

START · line 19

```
void myInit()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-200, 200, -150, 150);
}
```

END of myInit() · line 26

4 myDisplay() draws the figure, part by part

START · line 27

```
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);

    // wall
    glColor3f(0.9, 0.9, 0.8);
    glBegin(GL_QUADS);
    glVertex2f(-200, -150);
    glVertex2f(200, -150);
    glVertex2f(200, 150);
    glVertex2f(-200, 150);
    glEnd();

    // clock face
    glColor3f(1.0, 1.0, 1.0);
    circleMidpoint(0, 0, 120, GL_LINES);
    glColor3f(0.0, 0.0, 0.0);
    glPointSize(4);
    circleMidpoint(0, 0, 120, GL_POINTS);

    // hour marks
    glBegin(GL_QUADS);
    glVertex2f(-5, 90);
    glVertex2f(5, 90);
    glVertex2f(5, 112);
    glVertex2f(-5, 112);
    glVertex2f(-5, -112);
    glVertex2f(5, -112);
    glVertex2f(5, -90);
    glVertex2f(-5, -90);
    glVertex2f(90, -5);
```

```
    glVertex2f(112, -5);
    glVertex2f(112, 5);
    glVertex2f(90, 5);
    glVertex2f(-112, -5);
    glVertex2f(-90, -5);
    glVertex2f(-90, 5);
    glVertex2f(-112, 5);
    glEnd();
    glPointSize(8);
    glBegin(GL_POINTS);
    glVertex2f(50, 87);
    glVertex2f(87, 50);
    glVertex2f(87, -50);
    glVertex2f(50, -87);
    glVertex2f(-50, -87);
    glVertex2f(-87, -50);
    glVertex2f(-87, 50);
    glVertex2f(-50, 87);
    glEnd();

    // hands
    glLineWidth(6);
    glBegin(GL_LINES);
    glVertex2f(0, 0);
    glVertex2f(0, 70);
    glEnd();
    glLineWidth(3);
    glBegin(GL_LINES);
    glVertex2f(0, 0);
    glVertex2f(85, 0);
    glEnd();
    glColor3f(1.0, 0.0, 0.0);
    glLineWidth(1);
    glBegin(GL_LINES);
    glVertex2f(0, 0);
    glVertex2f(-60, -60);
    glEnd();

    // center
    glPointSize(12);
    glBegin(GL_POINTS);
    glVertex2f(0, 0);
    glEnd();

    glFlush();
}
```

END of myDisplay() · line 103

5 **circleMidpoint()** midpoint circle algorithm (Lec 5) START · line 104

```
void circleMidpoint(int xc, int yc, int radius, GLenum mode)
{
    int x = 0;
    int y = radius;
    int p0 = 1 - radius;
    circleplot(xc, yc, x, y, mode);
    while (x < y)
```

```
111     {
112         x++;
113         if (p0 < 0)
114             p0 = p0 + 2 * x + 1;
115         else
116         {
117             y--;
118             p0 = p0 + 2 * x -2*y + 1;
119         }
120         circleplot(xc, yc, x, y, mode);
121     }
122 }
```

END of circleMidpoint() · line 122

6 `circleplot()` plots the 8 symmetric points

START · line 123

```
123 void circleplot(int xc, int yc, int x, int y, GLenum mode)
124 {
125     glBegin(mode);
126     glVertex2f(xc + x, yc + y);
127     glVertex2f(xc - x, yc + y);
128     glVertex2f(xc + x, yc - y);
129     glVertex2f(xc - x, yc - y);
130     glVertex2f(xc + y, yc + x);
131     glVertex2f(xc - y, yc + x);
132     glVertex2f(xc + y, yc - x);
133     glVertex2f(xc - y, yc - x);
134     glEnd();
135 }
```

END of circleplot() · line 135

الفكرة البرنامج ملف واحد مقسوم functions: كل function بتبدأ عند START وتخلص عند END. وجوه myDisplay كل جزء من الصورة بيبدأ بـ comment ملوّن.

CG / MODEL ANSWER / MODEL 09

Model Answer — Model 9

M9

Exam paper on page 30 · each answer shows the lecture page of its method

Q1

Source: the page is next to each answer

Question 1 — theory

a · Define

Clipping: Cutting vertices that will not be on the screen. Lec 1 · p24

Rasterization: Creating pixels for each part of the object. Lec 1 · p24

Polygon: A closed polyline where its start point equal to its last point. Lec 2 · p12

b · Compare Monochrome, gray-scale and color raster images Lec 2 · p22–24

Image	Pixel contains
Monochrome	One of two values (bi-level)
Gray-scale	A gray color depending on pixel depth (N bit → 2^N gray levels)
Color	A triple of red, green and blue values (true color = 24 bit)

c · True / False

1. ✓ **True** $R(\theta) \cdot R(\varphi) = R(\varphi) \cdot R(\theta)$. Lec 6 · p26
2. ✗ **False** The inverse of the scaling $S(s_x, s_y)$ is $S(-s_x, -s_y)$.
   Correct: $S^{-1}(s_x, s_y) = S(1/s_x, 1/s_y)$.
   Lec 6 · p24
3. ✗ **False** Emissive displays use optical effects to convert light from some other source.
   Correct: That is non-emissive displays; emissive displays convert electrical energy into light.
   Lec 2 · p27
4. ✗ **False** The vertex shader adds values for each pixel (coloring object shape).
   Correct: That is the fragment shader; the vertex shader defines object vertices to move the object around.
   Lec 1 · p24
5. ✓ **True** In the midpoint circle algorithm $P = x^2 + y^2 - R^2$; if $P = 0$ the point is on the circle. Lec 5 · p9

d · Complete

1. Shear (skewing) deforms shape like _____. → **shifted slices** Lec 6 · p11
2. Bresenham's method is described for a line segment with a positive slope less than _____. → **one**
   Lec 4 · p25
3. For a circle with center (x_c, y_c): $x = x_c +$ _____ and $y = y_c +$ _____. → **cos θ * r · sin θ * r** Lec 5 · p8

e · Explain 1) Why is 2D learned before 3D? Lec 1 · p18

- 2D is a good stepping stone towards 3D; many issues are much easier to understand in 2D.
- No need to simulate lights, cameras, etc.
- 2D is still important and the most common use of computer graphics, as in advertising.

2) In 3D, how do we scale an object w.r.t. a fixed position (x_f, y_f, z_f)? Lec 6 · p42

- Translate the fixed point to the origin.
- Apply the scaling transformation relative to the coordinate origin.
- Translate the fixed point back to its original position.

f · Calculate

1. If we use 8 bits pixel value in a 24-bit lookup table, find the LUT size.
   → **$(2^8 \times 24)/8 = 768$ bytes** Lec 2 · p38
2. Bresenham's line from (10, 0) to (20, 8): find the first four pixels after (10, 0).
   → **p0 = 16 − 10 = 6 → (11, 1) · p = 2 → (12, 2) · p = −2 → (13, 2) · p = 14 → (14, 3)** Lec 4 · p28–30

الفكرة الإجابة بجمل المحاضرة نفسها: التعريف جملة، والمقارنة جدول، والـ False لازم يتصحح.

Q2-1 Source: Lec 5 · p16–23 · Graphics_Sec 6 · p2

Question 2 (1) — Midpoint ellipse · $r_x = 6$ and $r_y = 8$

HOW TO SOLVE

1. Compute r_x^2, r_y^2 and $p1_0$.
2. Region 1: fill the table (increment x) until $2r_y^2x \geq 2r_x^2y$.
3. Compute $p2_0$ from the last point of region 1.
4. Region 2: fill the table (decrement y) until y = 0.
5. Plot the points.

$r_x^2 = 36$, $r_y^2 = 64$ · initial point (0, 8)
$p1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4}r_x^2 = 64 - 36\times8 + \frac{1}{4}\times36 = -215$

Region 1

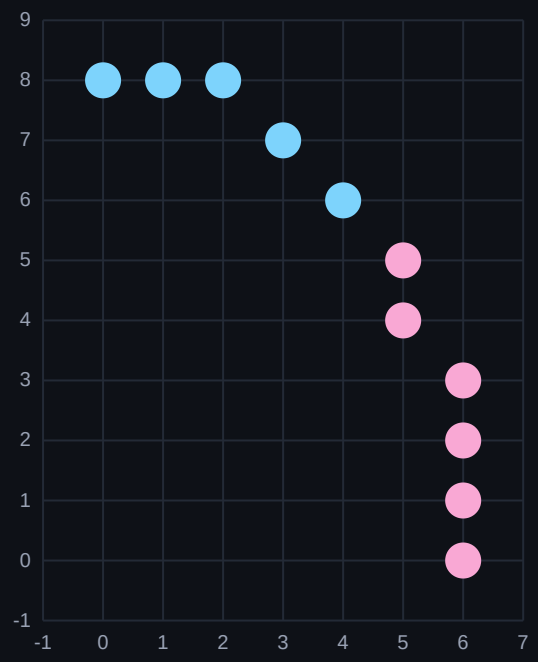
Pk_region1	x	y	$2r_y^2x$	$2r_x^2y$
	0	8	0	576
−215	1	8	128	576
−23	2	8	256	576
297	3	7	384	504
241	4	6	512	432

Region 1 stops when $2r_y^2x \geq 2r_x^2y$ → at (4, 6): 512 ≥ 432
Initial point of region 2 = last point of region 1 (4, 6) · $p2_0 = r_y^2(x_0 + \frac{1}{2})^2 + r_x^2(y_0 - 1)^2 - r_x^2r_y^2 = 64\times(4.5)^2 + 36\times(5)^2 - 36\times64 = -108$

Region 2

Pk_region2	x	y	$2r_y^2x$	$2r_x^2y$
	4	6	512	432
−108	5	5	640	360
208	5	4	640	288
−44	6	3	768	216
544	6	2	768	144
436	6	1	768	72
400	6	0		

Region 1 (blue) · Region 2 (pink)



الفكرة Region 1 بنزوّد x لحد ما $x_y^2 2r$ يبقى ≤ $y_x^2 2r$، وبعدين Region 2 بنقلّل y لحد الصفر.

Q2-2

Source: Lec 6 · p20–33 · Lec 1 · p22

Question 2 (2) — P′ = R(60) · S(2, 0.5) · T(−3, 2) · P

Read from right to left — the matrix nearest to P is applied first:

1. **T(−3, 2):** Translate the object 3 units to the left and 2 units up.
2. **S(2, 0.5):** Scale around the origin: x is doubled (stretched horizontally) and y is halved (squashed vertically).
3. **R(60):** Rotate 60 degree counter-clockwise around the origin.

So the object is moved by (−3, 2), stretched horizontally and squashed vertically about the origin, then rotated 60° about the origin.

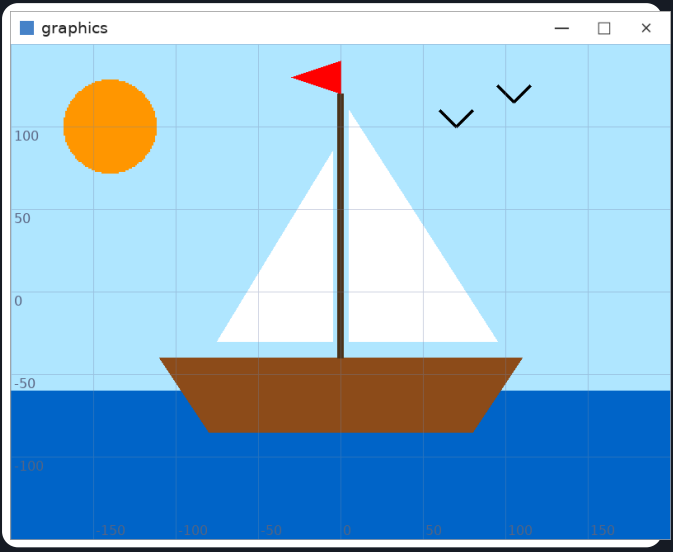
$$\begin{vmatrix} \cos 60 & -\sin 60 & 0 \\ \sin 60 & \cos 60 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 2 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & -3 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{vmatrix}$$

الفكرة اقرا الـ matrices من اليمين للشمال، وكل matrix = جملة واحدة بتوصف اللي حصل للشكل.

Q3

Source: main / myInit: Lec 3 · p10 · circleMidpoint: Lec 5 · p14

Question 3 — program



The output with the world-coordinate grid of gluOrtho2D(−200, 200, −150, 150) — read the vertices from it. Any coordinates that give the same figure are correct.

PROGRAM MAP — one file, written in this order

1	Headers & prototypes	libraries + declare the functions	lines 1–6
2	main()	create the window and start the program	lines 7–18
3	myInit()	background color + coordinate system	lines 19–26
4	myDisplay()	draws the figure, part by part	lines 27–104
5	circleMidpoint()	midpoint circle algorithm (Lec 5)	lines 105–123
6	circleplot()	plots the 8 symmetric points	lines 124–136

1 Headers & prototypes — libraries + declare the functions

START · line 1

```
1  #include <windows.h>
2  #include <GL/glut.h>
3  void myInit();
4  void myDisplay();
5  void circleMidpoint(int xc, int yc, int radius, GLenum mode);
6  void circleplot(int xc, int yc, int x, int y, GLenum mode);
```

END of Headers & prototypes · line 6

2 main() — create the window and start the program

START · line 7

```
7  int main(int argc, char** argv)
8  {
9      glutInit(&argc, argv);
10     glutInitDisplayMode(GLUT_RGB);//set display mode
```

```
    glutInitWindowSize(400, 300);//set window size
    glutInitWindowPosition(100, 150);//set window position
    glutCreateWindow("graphics");
    myInit();
    glutDisplayFunc(myDisplay);
    glutMainLoop();
    return 0;
}
```

END of main() · line 18

3 myInit() background color + coordinate system

START · line 19

```
void myInit()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-200, 200, -150, 150);
}
```

END of myInit() · line 26

4 myDisplay() draws the figure, part by part

START · line 27

```
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);

    // sky
    glColor3f(0.7, 0.9, 1.0);
    glBegin(GL_QUADS);
    glVertex2f(-200, -150);
    glVertex2f(200, -150);
    glVertex2f(200, 150);
    glVertex2f(-200, 150);
    glEnd();

    // sun
    glColor3f(1.0, 0.6, 0.0);
    circleMidpoint(-140, 100, 28, GL_LINES);

    // birds
    glColor3f(0.0, 0.0, 0.0);
    glLineWidth(2);
    glBegin(GL_LINE_STRIP);
    glVertex2f(60, 110);
    glVertex2f(70, 100);
    glVertex2f(80, 110);
    glEnd();
    glBegin(GL_LINE_STRIP);
    glVertex2f(95, 125);
    glVertex2f(105, 115);
    glVertex2f(115, 125);
    glEnd();

```

```
    // sea
    glColor3f(0.0, 0.4, 0.8);
    glBegin(GL_QUADS);
    glVertex2f(-200, -150);
    glVertex2f(200, -150);
    glVertex2f(200, -60);
    glVertex2f(-200, -60);
    glEnd();

    // hull
    glColor3f(0.6, 0.3, 0.1);
    glBegin(GL_POLYGON);
    glVertex2f(-110, -40);
    glVertex2f(110, -40);
    glVertex2f(80, -85);
    glVertex2f(-80, -85);
    glEnd();

    // mast
    glColor3f(0.3, 0.2, 0.1);
    glLineWidth(4);
    glBegin(GL_LINES);
    glVertex2f(0, -40);
    glVertex2f(0, 120);
    glEnd();

    // sails
    glColor3f(1.0, 1.0, 1.0);
    glBegin(GL_TRIANGLES);
    glVertex2f(5, -30);
    glVertex2f(5, 110);
    glVertex2f(95, -30);
    glVertex2f(-5, -30);
    glVertex2f(-5, 85);
    glVertex2f(-75, -30);
    glEnd();

    // flag
    glColor3f(1.0, 0.0, 0.0);
    glBegin(GL_TRIANGLES);
    glVertex2f(0, 120);
    glVertex2f(0, 140);
    glVertex2f(-30, 130);
    glEnd();

    glFlush();
}
```

END of myDisplay() · line 104

5 circleMidpoint() midpoint circle algorithm (Lec 5)

START · line 105

```
void circleMidpoint(int xc, int yc, int radius, GLenum mode)
{
    int x = 0;
    int y = radius;
    int p0 = 1 - radius;
    circleplot(xc, yc, x, y, mode);
```

```
111     while (x < y)
112     {
113         x++;
114         if (p0 < 0)
115             p0 = p0 + 2 * x + 1;
116         else
117         {
118             y--;
119             p0 = p0 + 2 * x -2*y + 1;
120         }
121         circleplot(xc, yc, x, y, mode);
122     }
123 }
```

END of circleMidpoint() · line 123

6 circleplot() plots the 8 symmetric points

START · line 124

```
124 void circleplot(int xc, int yc, int x, int y, GLenum mode)
125 {
126     glBegin(mode);
127     glVertex2f(xc + x, yc + y);
128     glVertex2f(xc - x, yc + y);
129     glVertex2f(xc + x, yc - y);
130     glVertex2f(xc - x, yc - y);
131     glVertex2f(xc + y, yc + x);
132     glVertex2f(xc - y, yc + x);
133     glVertex2f(xc + y, yc - x);
134     glVertex2f(xc - y, yc - x);
135     glEnd();
136 }
```

END of circleplot() · line 136

الفكرة البرنامج ملف واحد مقسوم functions: كل function بتبدأ عند START وتخلص عند END. وجوه myDisplay كل جزء من الصورة بيبدأ بـ comment ملوّن.

CG / MODEL ANSWER / MODEL 10

Model Answer — Model 10

M10

Exam paper on page 32 · each answer shows the lecture page of its method

Q1

Source: the page is next to each answer

Question 1 — theory

a · Define

Circle: A set of all points in a plane that are all an equal distance (the radius) from a single point, the center. Lec 5 · p3

Attributes of a graphic primitive: The characteristics that affect how it appears, such as color, style and thickness. Lec 2 · p10

Color palette: The set of possible colors displayed by the system. Lec 2 · p35

b · Compare Hardware level and software level of interactive computer graphics Lec 1 · p17

Hardware level	Software level
The computer receives input from interactive devices and shows images on display devices	Involves three components: application program, application model, graphics library (API)

c · True / False

1. ✓ **True** The inverse translation $T^{-1}(t_x, t_y) = T(-t_x, -t_y)$. Lec 6 · p22
2. ✗ **False** Drawing a circle with $y = \sqrt{(r^2 - x^2)}$ gives uniform pixel spacing.
   Correct: This method needs lots of computation, and gives non-uniform pixel spacing.
   Lec 5 · p5
3. ✓ **True** In DDA the method of round-off is also time-consuming. Lec 4 · p24
4. ✓ **True** In true color each DAC has eight input bits: 256 levels of each color, 16 million colors in total.
   Lec 2 · p35
5. ✗ **False** NDC are the coordinates mapped to the specific resolution of the output device (e.g. 1024 × 768).
   Correct: In NDC the coordinates are standardized between 0 and 1; device coordinates are mapped to the specific resolution.
   Graphics_SEC 3 · p1

d · Complete

1. Composite transformations are used to rotate or scale an object w.r.t. a point that is not the _____. → **origin**
   Lec 6 · p29
2. In a raster scan system the _____ takes care of the actual display process. → **scan controller** Lec 2 · p30
3. After setPalette(39, 17, 25, 4), to make the pixel (479, 532) glow in this color, the _____ is stored in the frame buffer. → **value 39 (drawDot(479, 532, 39))** Lec 2 · p37

e · Explain 1) Write the steps to rotate an object around a pivot point. Lec 6 · p30

- Translate the object so that the pivot point moves to the origin.
- Rotate around origin.
- Translate the object so that the pivot point is back to its original position.
- $P' = T(t_x, t_y) \cdot R(\theta) \cdot T(-t_x, -t_y) \cdot P$

2) List the five stages of the graphics pipeline. Graphics_SEC 3 · p1

- Model coordinates: object shapes are defined locally.
- World coordinates: models are positioned within a global scene.
- Viewing coordinates: the scene is projected onto a virtual screen from a camera's perspective.
- Normalized device coordinates (NDC): coordinates standardized between 0 and 1.
- Device coordinates: the image is mapped to the specific resolution of the output device.

f · Calculate

1. If we use a 10-bit pixel value in a 32-bit lookup table, find the entries and the size in bytes.
   → **2^{10} = 1,024 entries · (1024 × 32)/8 = 4,096 bytes** Graphics_SEC 3 · p2
2. Window (0, 2, 0, 1), viewport (40, 400, 60, 300). Find Sx and Sy.
   → **Sx = (400 − 40)/(2 − 0) = 180 · Sy = (300 − 60)/(1 − 0) = 240** Lec 3 · p5

الفكرة الإجابة بجمل المحاضرة نفسها: التعريف جملة، والمقارنة جدول، والـ False لازم يتصحح.

Q2-1

Source: Lec 4 · p19–23 · Graphics sec5 · p1

Question 2 (1) — DDA line · (14, 12) to (6, 7)

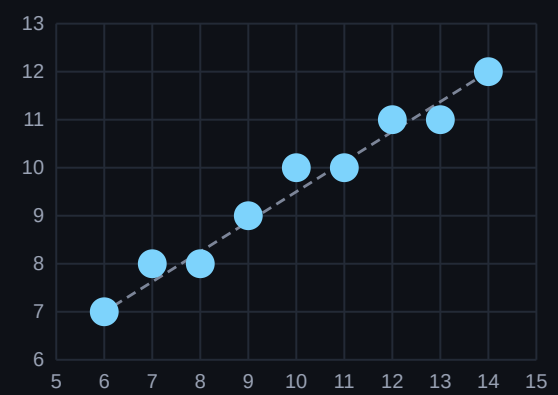
HOW TO SOLVE

1. Compute Δx, Δy, steps and m (or 1/m).
2. Decide the increments (+ or −) from the direction of the line.
3. Add the increments step by step and round each value.
4. Plot the pixels.

$\Delta x = -8, \Delta y = -5 \rightarrow |\Delta x| \geq |\Delta y|$ ($|m| \leq 1$) · $m = 0.62$ · Steps = 8 · $x_{k+1} = x_k - 1$ · $y_{k+1} = y_k - m$

K	x	y	(x, y)
0	13	11.38	(13,11)
1	12	10.75	(12,11)
2	11	10.12	(11,10)
3	10	9.5	(10,10)
4	9	8.88	(9,9)
5	8	8.25	(8,8)
6	7	7.62	(7,8)
7	6	7	(6,7)

Plotted pixels



الفكرة الـ coordinate اللي بيتغير أكتر بيمشي بخطوة 1، والتاني بيزيد بالكسر ويتقرّب.

Q2-2

Source: Lec 6 · p20–33 · Lec 1 · p22

Question 2 (2) — P′ = T(1, 2) · R(30) · R(60) · T(−1, −2) · P

Read from right to left — the matrix nearest to P is applied first:

1. **T(−1, −2):** Translate the object so that the pivot point (1, 2) moves to the origin.
2. **R(60):** Rotate 60 degree counter-clockwise around the origin.
3. **R(30):** Rotate another 30 degree counter-clockwise around the origin.
4. **T(1, 2):** Translate the object so that the pivot point is back to its original position.

Rotation angles add: R(30)·R(60) = R(90). So the object is rotated 90° around the point (1, 2).

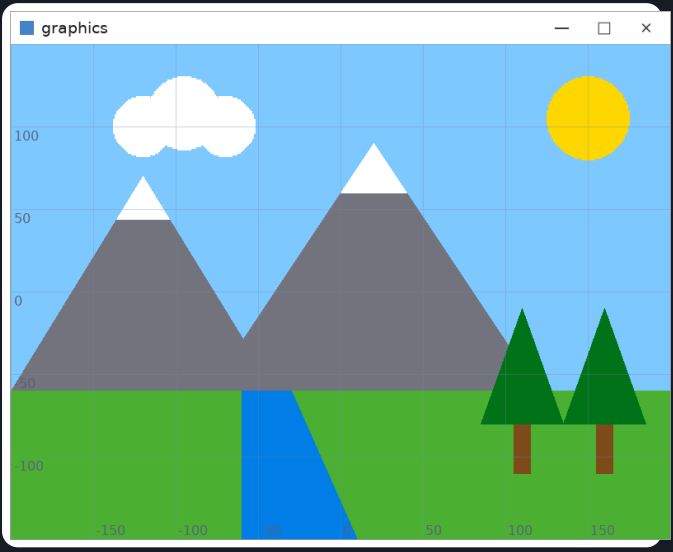
$$\begin{vmatrix} 1 & 0 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos 30 & -\sin 30 & 0 \\ \sin 30 & \cos 30 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos 60 & -\sin 60 & 0 \\ \sin 60 & \cos 60 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & -1 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{vmatrix}$$

الفكرة اقرا الـ matrices من اليمين للشمال، وكل matrix = جملة واحدة بتوصف اللي حصل للشكل.

Q3

Source: main / myInit: Lec 3 · p10 · circleMidpoint: Lec 5 · p14

Question 3 — program



The output with the world-coordinate grid of gluOrtho2D(−200, 200, −150, 150) — read the vertices from it. Any coordinates that give the same figure are correct.

PROGRAM MAP — one file, written in this order

1	Headers & prototypes	libraries + declare the functions	lines 1–6
2	main()	create the window and start the program	lines 7–18
3	myInit()	background color + coordinate system	lines 19–26
4	myDisplay()	draws the figure, part by part	lines 27–113
5	circleMidpoint()	midpoint circle algorithm (Lec 5)	lines 114–132
6	circleplot()	plots the 8 symmetric points	lines 133–145

1 Headers & prototypes — libraries + declare the functions

START · line 1

```
1  #include <windows.h>
2  #include <GL/glut.h>
3  void myInit();
4  void myDisplay();
5  void circleMidpoint(int xc, int yc, int radius, GLenum mode);
6  void circleplot(int xc, int yc, int x, int y, GLenum mode);
```

END of Headers & prototypes · line 6

2 main() — create the window and start the program

START · line 7

```
7  int main(int argc, char** argv)
8  {
9      glutInit(&argc, argv);
10     glutInitDisplayMode(GLUT_RGB);//set display mode
```

```
11      glutInitWindowSize(400, 300);//set window size
12      glutInitWindowPosition(100, 150);//set window position
13      glutCreateWindow("graphics");
14      myInit();
15      glutDisplayFunc(myDisplay);
16      glutMainLoop();
17      return 0;
18  }
```

END of main() · line 18

3 myInit() background color + coordinate system

START · line 19

```
19  void myInit()
20  {
21      glClearColor(1.0, 1.0, 1.0, 0.0);
22      glColor3f(0.0f, 0.0f, 0.0f);
23      glMatrixMode(GL_PROJECTION);
24      glLoadIdentity();
25      gluOrtho2D(-200, 200, -150, 150);
26  }
```

END of myInit() · line 26

4 myDisplay() draws the figure, part by part

START · line 27

```
27  void myDisplay()
28  {
29      glClear(GL_COLOR_BUFFER_BIT);
30
31      // sky
32      glColor3f(0.5, 0.8, 1.0);
33      glBegin(GL_QUADS);
34      glVertex2f(-200, -150);
35      glVertex2f(200, -150);
36      glVertex2f(200, 150);
37      glVertex2f(-200, 150);
38      glEnd();
39
40      // sun
41      glColor3f(1.0, 0.8, 0.0);
42      circleMidpoint(150, 105, 25, GL_LINES);
43
44      // cloud
45      glColor3f(1.0, 1.0, 1.0);
46      circleMidpoint(-120, 100, 18, GL_LINES);
47      circleMidpoint(-95, 108, 22, GL_LINES);
48      circleMidpoint(-70, 100, 18, GL_LINES);
49
50      // mountains
51      glColor3f(0.5, 0.5, 0.5);
52      glBegin(GL_TRIANGLES);
53      glVertex2f(-200, -60);
54      glVertex2f(-40, -60);
55      glVertex2f(-120, 70);
56      glVertex2f(-80, -60);
57      glVertex2f(120, -60);
```

```
    glVertex2f(20, 90);
    glEnd();

    // snow caps
    glColor3f(1.0, 1.0, 1.0);
    glBegin(GL_TRIANGLES);
    glVertex2f(-136, 44);
    glVertex2f(-104, 44);
    glVertex2f(-120, 70);
    glVertex2f(0, 60);
    glVertex2f(40, 60);
    glVertex2f(20, 90);
    glEnd();

    // grass
    glColor3f(0.3, 0.7, 0.2);
    glBegin(GL_QUADS);
    glVertex2f(-200, -150);
    glVertex2f(200, -150);
    glVertex2f(200, -60);
    glVertex2f(-200, -60);
    glEnd();

    // trees
    glColor3f(0.5, 0.3, 0.1);
    glBegin(GL_QUADS);
    glVertex2f(105, -110);
    glVertex2f(115, -110);
    glVertex2f(115, -80);
    glVertex2f(105, -80);
    glVertex2f(155, -110);
    glVertex2f(165, -110);
    glVertex2f(165, -80);
    glVertex2f(155, -80);
    glEnd();
    glColor3f(0.0, 0.5, 0.1);
    glBegin(GL_TRIANGLES);
    glVertex2f(85, -80);
    glVertex2f(135, -80);
    glVertex2f(110, -10);
    glVertex2f(135, -80);
    glVertex2f(185, -80);
    glVertex2f(160, -10);
    glEnd();

    // river
    glColor3f(0.0, 0.5, 0.9);
    glBegin(GL_POLYGON);
    glVertex2f(-60, -60);
    glVertex2f(-30, -60);
    glVertex2f(10, -150);
    glVertex2f(-60, -150);
    glEnd();

    glFlush();
}
```

END of myDisplay() · line 113

5 `circleMidpoint()` midpoint circle algorithm (Lec 5)

START · line 114

```
114 void circleMidpoint(int xc, int yc, int radius, GLenum mode)
115 {
116     int x = 0;
117     int y = radius;
118     int p0 = 1 - radius;
119     circleplot(xc, yc, x, y, mode);
120     while (x < y)
121     {
122         x++;
123         if (p0 < 0)
124             p0 = p0 + 2 * x + 1;
125         else
126         {
127             y--;
128             p0 = p0 + 2 * x -2*y + 1;
129         }
130         circleplot(xc, yc, x, y, mode);
131     }
132 }
```

END of circleMidpoint() · line 132

6 `circleplot()` plots the 8 symmetric points

START · line 133

```
133 void circleplot(int xc, int yc, int x, int y, GLenum mode)
134 {
135     glBegin(mode);
136     glVertex2f(xc + x, yc + y);
137     glVertex2f(xc - x, yc + y);
138     glVertex2f(xc + x, yc - y);
139     glVertex2f(xc - x, yc - y);
140     glVertex2f(xc + y, yc + x);
141     glVertex2f(xc - y, yc + x);
142     glVertex2f(xc + y, yc - x);
143     glVertex2f(xc - y, yc - x);
144     glEnd();
145 }
```

END of circleplot() · line 145

الفكرة البرنامج ملف واحد مقسوم functions: كل function بتبدأ عند START وتخلص عند END. وجوه myDisplay كل جزء من الصورة بيبدأ بـ comment ملوّن.

CG / PART 4 / QUICK ANSWER KEY

Quick Answer Key

04

Question 1 — True/False and Complete of all models, for fast checking

MODEL	TRUE / FALSE (1–3)	COMPLETE	FULL ANSWER PAGE
Model 1	✗ ✓ ✗ ✓ ✗	what · where | 2^N | World · NDC	35
Model 2	✗ ✓ ✗ ✓ ✗	how objects appears (color, texture or shape) · how object moves | start point · last point | weight · orientation	44
Model 3	✗ ✓ ✗ ✗ ✓	polyline · text · filled regions · raster images | center | getPixel	52
Model 4	✗ ✓ ✗ ✓ ✗	coordinate position (x, y) · selection of a part of the screen | x · y | smartphones and tablets · smart TVs	59
Model 5	✓ ✗ ✓ ✓ ✓	clipping region | midpoint (ratios of distances) | $(y_{end} - y_0)/(x_{end} - x_0)$ · $y_0 - m \cdot x_0$	67
Model 6	✗ ✓ ✗ ✓ ✗	color · thickness · type (dashed, solid, dotted) | $R(\theta + \varphi)$ | lesser	75
Model 7	✗ ✓ ✗ ✓ ✗	physics · mathematics | center | value of the pixel	83
Model 8	✗ ✓ ✓ ✓ ✗	scalable vector graphics (SVG) | 24 · 8 | $(x_k + 1, y_k)$ · $(x_k + 1, y_k + 1)$	90
Model 9	✓ ✗ ✗ ✗ ✓	shifted slices | one | cos θ * r · sin θ * r	97
Model 10	✓ ✗ ✓ ✓ ✗	origin | scan controller | value 39 (drawDot(479, 532, 39))	105

Score yourself

- Question 1: 1 point for each part a, b, c, d, e (5 points) — the definition, comparison and explanation must use the lecture sentences.
- Question 2: 5 points for the algorithm table + plot, 5 points for the step-by-step description.
- Question 3: 5 points — main + myInit + myDisplay drawing every part of the figure.

لو غلطت في سؤال، ارجع لنفس النوع في الـ Solving Guide (Part 1)، وللنظري ارجع للمحاضرة والصفحة المكتوبة جنب الإجابة.